

ENTERPRISE ARCHITECTURE PACKAGE

Autonomous AI Job Application Platform

Architecture & Design Reference v1.0

AUTHOR	ROLE	DATE	VERSION	CLASSIFICATION
Vikram Deshpande	Sr. TPM / AI Solutions Architect	July 2026	v1.0 — Initial Release	Internal — Confidential
STATUS				
Architecture Review Draft				

Table of Contents

Autonomous AI Job Application Platform — Enterprise Architecture Package v1.0 | Vikram Deshpande | July 2026

2	Executive Summary	3	Multi-Agent Orchestration	26	11	AI Memory Architecture	16	Technology Stack Recommendations
3	Product Vision & Strategy	7	Sequence Diagrams	29	12	Knowledge Graph	17	Implementation Roadmap
4	Enterprise Architecture — C4 Model	8	Resume Tailoring Pipeline	33	13	Security Architecture	18	Decision Trees
Context, Container, Component, Deployment				9	14	Error Handling & Recovery	19	State Machines
5	AI Agent Architecture (20 Agents)	10	ATS Optimization Engine	36	15	Analytics & KPIs	20	Appendix
		10	Data Model & ERD	45				

Executive Summary

Vision Statement

The **Autonomous AI Job Application Platform** is an intelligent career agent system that continuously discovers, evaluates, tailors, applies, and learns — acting as a tireless professional advocate for its user. Built for the modern AI-first job market, it transforms what is currently an exhausting, manual, and largely ineffective process into a precision-engineered pipeline that delivers measurable outcomes: more interviews, higher offer rates, and dramatically less time spent on administrative job-search mechanics.

This platform is not a simple automation script. It is a sophisticated multi-agent orchestration system combining state-of-the-art LLMs, semantic search, browser automation, and continuous reinforcement learning into a coherent, auditable, and responsible product. Every action is grounded in evidence from the user's verified portfolio, every claim is traceable to a source of record, and every submission meets a quantifiable quality bar before it leaves the system.

Problem Statement

Job searching at a senior technical level is paradoxically inefficient. A candidate of Vikram Deshpande's calibre — 15+ years of delivery leadership, AI/ML architecture, and enterprise program management across ATO, ANZ, NAB,

MYOB, Microsoft, and Telstra — should be spending time on high-value activities: preparing for interviews, networking strategically, and evaluating opportunities. Instead, the reality is:

- Manually tailoring resumes for each application takes 45-90 minutes per submission
- ATS systems reject up to 75% of qualified candidates for keyword mismatch rather than capability gaps
- Cover letter generation is repetitive, yet each must be evidence-specific and company-aware
- Tracking applications across LinkedIn, SEEK, Indeed, and direct portals requires constant context switching
- Recruiter follow-up and calendar management create an additional cognitive load
- Without analytics, there is no feedback loop to improve the application strategy over time

Solution Overview

The platform addresses every problem in the job-search lifecycle through a modular multi-agent architecture. The system operates as follows: a **Job Discovery Agent** continuously monitors job boards and APIs; a **Planner Agent** evaluates fit against the user's profile; a **Resume Tailoring Agent** produces ATS-optimised resumes without changing a single pixel of the source formatting; a **Cover Letter Agent** generates evidence-backed, company-specific letters; an **Application Agent** submits the package via browser automation with human approval gates; and a **Learning Agent** continuously refines the strategy based on outcomes. Every step is monitored by a **Compliance Agent** that guarantees no fabrication, no PII leakage, and full audit trail.

Business Goals

- Reduce active job-search time from 20+ hours per week to fewer than 4 hours
- Increase interview conversion rate from typical 2-3% to 8-12% (3x improvement)
- Submit 5-10 high-quality applications per day versus 1-2 manually
- Achieve 85%+ ATS pass rate for every submitted resume
- Build a continuously improving application strategy that compounds over time
- Maintain 100% factual accuracy — zero fabricated achievements or skills

ROI Analysis

Metric	Manual Baseline	Platform Target	Improvement
Applications per day	1-2	5-10	5x
Hours per application	1.5 hrs	8 min	92% reduction
ATS pass rate	25-40%	85%+	3x
Interview conversion	2-3%	8-12%	4x
Time-to-first-interview	4-6 weeks	1-2 weeks	75% faster
Weekly active search hours	20+ hrs	4 hrs	80% reduction

Key Performance Indicators

5-10 Applications / Day	85%+ ATS Pass Rate	3x Interview Rate	80% Time Saved
----------------------------	-----------------------	----------------------	-------------------

92%

EFFORT REDUCTION (APPLY)

100%

FACTUAL ACCURACY

20

SPECIALISED AI AGENTS

16wk

BUILD ROADMAP

Future Roadmap Summary: Phase 1 (Weeks 1-4) delivers core infrastructure and resume pipeline. Phase 2 (Weeks 5-8) adds full agent intelligence and ATS optimization. Phase 3 (Weeks 9-12) delivers end-to-end automation including application submission and interview scheduling. Phase 4 (Weeks 13-16) completes the Learning Agent, Analytics Dashboard, and production hardening. The platform is designed to scale from single-user to enterprise team deployment.

Product Vision & Strategy

Mission

To empower elite technical professionals with an AI career agent that competes on their behalf — matching their genuine capabilities to the right opportunities with precision, evidence, and integrity. The platform acts as an always-on career partner that knows the user's portfolio deeply, speaks their professional language fluently, and never overstates or fabricates.

Vision

To become the definitive autonomous career management platform for senior technical professionals in AI, engineering, and program management — where the days of manually tailoring resumes, chasing job boards, and drafting cover letters are replaced by a platform that handles the mechanics while the human focuses on the strategic and relational aspects of career advancement.

Core Values

Evidence-Grounded

Every claim, every achievement, every skill assertion is traceable to a verifiable source: GitHub repositories, portfolio projects, performance metrics, or employment records. No fabrication, ever.

Portfolio-Integrated

The system treats the user's portfolio website, GitHub profile, and LinkedIn as primary data sources. Applications are enriched with real project evidence, quantifiable metrics, and live demonstration links.

Human-in-the-Loop

High-confidence decisions are automated; ambiguous situations escalate to the human. The user maintains ultimate control over every submission, with a clear approval workflow and full audit trail.

Continuous Learning

The platform learns from every interaction, outcome, and feedback signal. Application strategies improve over time through reinforcement from interview rates, recruiter responses, and offer outcomes.

Target Audience

The primary audience is **senior technical professionals** with 10-20+ years of experience who are actively or passively seeking roles. Specifically:

- **Technical Program Managers** (TPM, SRTPM) managing multi-million dollar portfolios
- **AI/ML Solutions Architects** transitioning from delivery to design roles
- **Senior Delivery Leads** at Tier 1 financial institutions and government agencies
- **Scrum Masters and Agile Practitioners** with deep technical domains
- **Polyglot Engineers** with rare cross-functional expertise (e.g., Mainframe + AI)

Competitive Differentiation

PLATFORM	WHAT IT DOES	OUR ADVANTAGE
Teal HQ	Job tracking and basic tailoring suggestions	Full autonomous submission, 20 specialised agents, evidence-grounded
Jobscan	ATS keyword scoring and optimisation	End-to-end pipeline from discovery through to offer; ATS is just one component

LazyApply	Mass auto-apply with static resume	Quality over quantity — every application is tailored, verified, and evidence-backed
Simplify.jobs	Autofill job forms and track applications	Semantic resume tailoring, portfolio integration, multi-agent orchestration
Resume.io / Kickresume	Resume templates and AI suggestions	Format preservation, portfolio-sourced evidence, dynamic ATS scoring per JD

Unique Selling Proposition

Evidence-Grounded, Portfolio-Integrated, Never Fabricates. Unlike mass-apply tools that blast the same static resume to hundreds of jobs, this platform tailors every application with verified, specific achievements drawn directly from the user's portfolio, GitHub repositories, and employment history. The ATS Optimization Engine ensures every submission meets a quantifiable quality bar. The Compliance Agent enforces zero-fabrication at every stage. The result: fewer applications, higher quality, dramatically better outcomes.

The 92% Efficiency Precedent: Vikram's own ATO work — achieving 92% effort reduction in test-evidence automation — is the proof of concept for this platform's design philosophy. The same principles (structured evidence capture, automated pipeline orchestration, quality validation gates) are now applied to the job application domain.

Enterprise Architecture — C4 Model

The C4 model (Context, Container, Component, Deployment) provides a hierarchical, layered view of the platform architecture at increasing levels of detail. This section presents all four levels, each intended for a different audience: executives (Context), architects (Container), developers (Component), and DevOps engineers (Deployment).

Level 1 — System Context Diagram

The System Context diagram shows the platform as a single cohesive system and its relationships with the external actors and systems it interacts with. The user interacts with the platform via a web dashboard; the platform in turn interacts with all external services on the user's behalf.

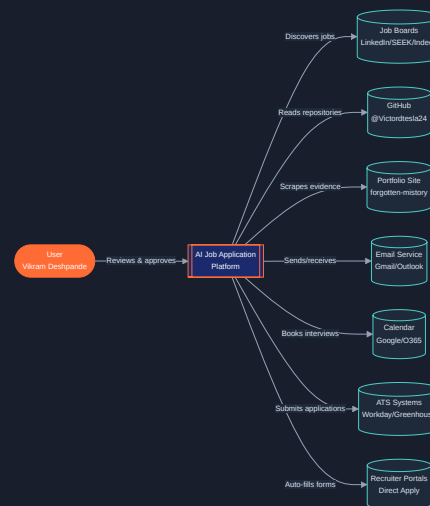
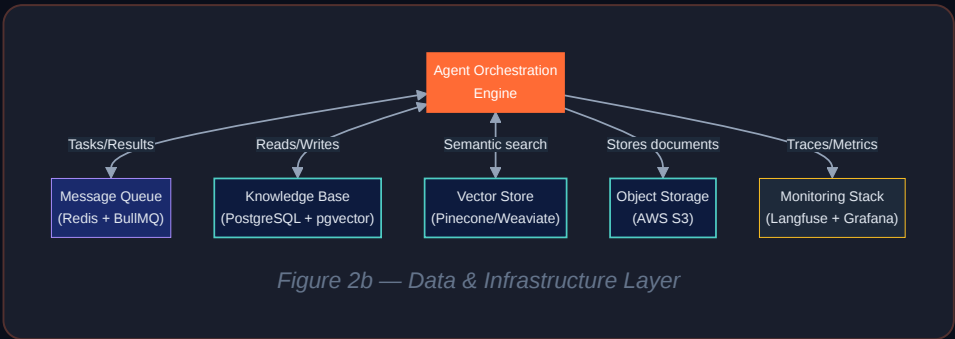
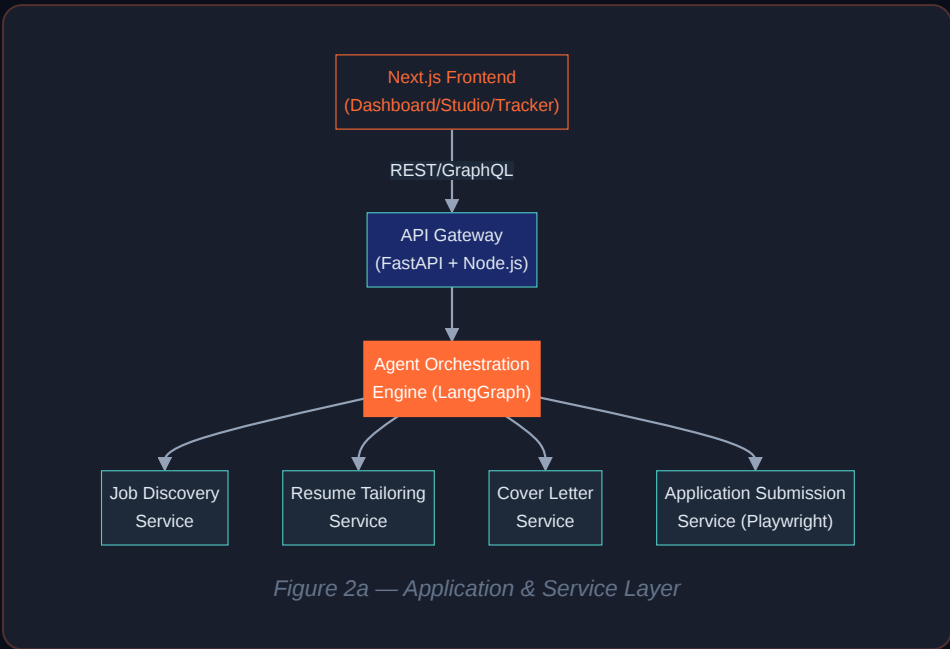


Figure 1 — C4 Level 1: System Context. The platform acts as autonomous intermediary between the user and all external career systems.

Level 2 — Container Diagram

The Container diagram decomposes the platform into its major deployable units. Containers communicate via REST APIs, GraphQL subscriptions, and the message queue. The Agent Orchestration Engine is the intelligence hub; all specialized agents run as workers pulling tasks from the queue.



Level 3 — Component Diagram (Agent Orchestration Engine)

The Agent Orchestration Engine is the cognitive core of the platform. It contains the Supervisor, Planner, and Executor components plus the supporting infrastructure for memory, tools, workflow, and human oversight.

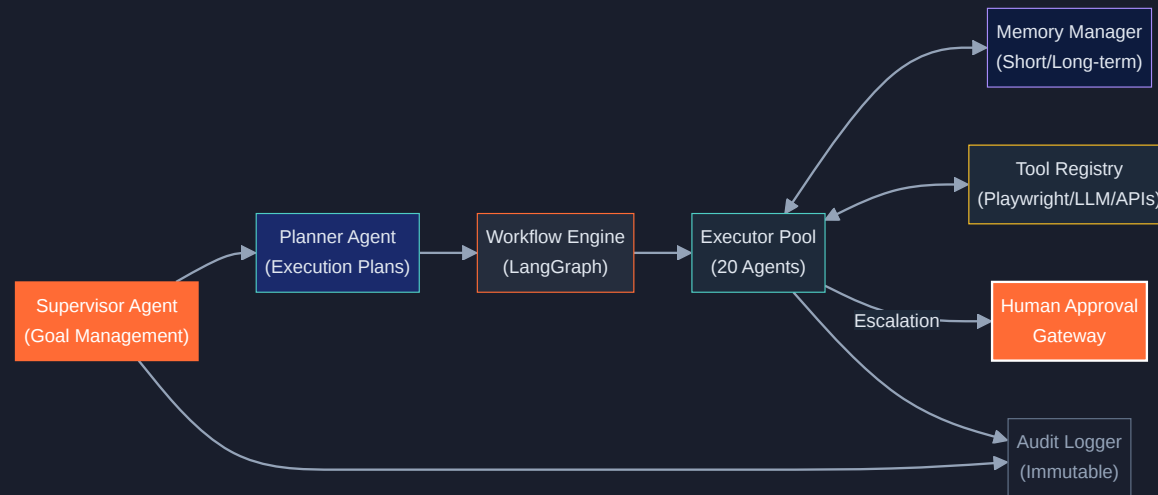


Figure 3 — C4 Level 3: Component Diagram of the Agent Orchestration Engine.

Level 4 — Deployment Diagram

The platform is deployed on AWS using Kubernetes (EKS) with separate namespaces for each logical layer. The CI/CD pipeline automates all deployments from GitHub commits through to production, with environment promotion gates.

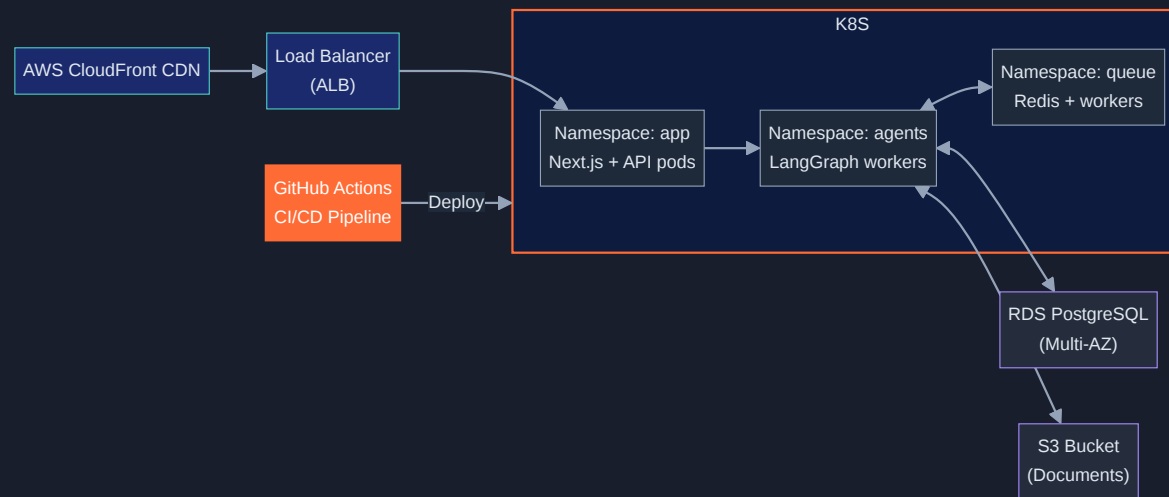


Figure 4 — C4 Level 4: Deployment Diagram on AWS EKS with namespaced Kubernetes workloads.

Architecture Principles

Separation of Concerns

Each agent has a single, well-defined responsibility. The Supervisor orchestrates; it does not execute. The Resume Tailoring Agent tailors; it does not submit. This prevents tight coupling and enables independent scaling and testing of each agent.

Event-Driven Messaging

All inter-agent communication flows through the Redis message queue. Agents subscribe to task queues and publish results as events. This decouples producers from consumers, enables async processing, and provides natural dead-letter queue support for failure handling.

Immutable Audit Trail

Every agent action — read, write, decision, or escalation — is logged to an immutable audit store. This enables full forensic replay of any application history, supports compliance verification, and provides the training data for the Learning Agent.

AI Agent Architecture

The platform comprises **20 specialised AI agents**, each with a clearly defined responsibility, input/output contract, tool access, memory scope, decision logic, and failure handling strategy. Agents communicate asynchronously via the Redis message queue and share state through the PostgreSQL knowledge base. All agents are implemented as LangGraph nodes within a directed acyclic graph workflow, enabling parallel execution, conditional branching, and human-in-the-loop gates.

Agent Design Principles: Single Responsibility — one agent, one function. Verifiable Outputs — every agent produces a confidence score alongside its output. Human Escalation — any confidence below threshold triggers an approval gate. Idempotent Execution — agents can be retried safely without side effects. Full Audit — every invocation is logged with inputs, outputs, timestamps, and reasoning.

Core Execution Agents (1-7)

01

Job Discovery Agent

Continuous job board surveillance and ranking

RESPONSIBILITIES
Crawls job boards (LinkedIn, SEEK, Indeed), RSS feeds, and company careers APIs on a configurable schedule. De-duplicates listings. Scores relevance against user profile.

INPUTS
Search criteria, keyword filters, salary range, location, seniority level, excluded companies

OUTPUTS
Ranked job list with relevance score, JD text, company metadata, application URL, posting date

TOOLS
Playwright (browser), LinkedIn API, SEEK API, Indeed RSS, web scraper

MEMORY SCOPE
Long-term: previously seen jobs, applied companies. Short-term: current crawl session

DECISION LOGIC
Relevance score > 70%: queue for evaluation. 50-70%: human review. <50%: discard with reason logged.

FAILURE HANDLING
Retry with exponential backoff (3 attempts). If API rate-limited, switch to RSS fallback. Alert on consecutive failures exceeding 5.

02

Resume Tailoring Agent

JD-specific resume optimisation with format preservation

RESPONSIBILITIES
Tailors resume content to match the job description using semantic skill mapping and keyword insertion. Preserves exact source formatting — typography, spacing, columns, and margins unchanged.

INPUTS
Base resume (PDF), job description text, portfolio evidence library, keyword priority list

OUTPUTS
Tailored resume PDF, ATS score, diff report showing changes, confidence score

TOOLS
PyMuPDF (parser), python-docx, LLM (GPT-4o), ATS Scorer, python-docx formatter

DECISION LOGIC
Only proceed to submission if ATS score > 75%. Below threshold: escalate to human with gap report.

FAILURE HANDLING
If formatting breaks, revert to base resume and flag for manual tailoring. Log all formatting violations. Never submit a resume with layout corruption.

03

Cover Letter Agent

Evidence-backed, company-specific letter generation

RESPONSIBILITIES

Generates personalised cover letters anchored to portfolio evidence.

Researches company context, culture, and recent news to create compelling narrative.

OUTPUTS

Cover letter (Word + PDF), evidence citations, authenticity score

MEMORY SCOPE

Portfolio evidence library, previously written letters (for style consistency)

DECISION LOGIC + FAILURE

Compliance Agent validates every output before release — any unverifiable claim is removed or flagged. If evidence insufficient, escalate to human for additional context.

INPUTS

JD text, tailored resume, company research summary, portfolio evidence, tone preference

TOOLS

LLM (Claude 3.5 Sonnet), web researcher, company news scraper, python-docx

04

Portfolio Agent

Evidence library construction from portfolio and GitHub

RESPONSIBILITIES

Crawls portfolio website (forgottenmistory.web.app) and GitHub (@Victordtesla24) to build a structured evidence library. Extracts metrics, project descriptions, technologies, and verifiable outcomes.

OUTPUTS

Structured evidence library: projects, metrics, technologies, source URLs, screenshots

MEMORY SCOPE

Long-term semantic store; updates triggered by portfolio changes or GitHub pushes

DECISION LOGIC + FAILURE

Reconciles new evidence with existing store. Flags conflicting metrics for human review. On scrape failure: use cached version with staleness flag and schedule retry.

INPUTS

Portfolio URL, GitHub username, crawl depth, update frequency

TOOLS

Playwright, GitHub API v4 (GraphQL), web scraper, content extractor, image capturer

05

Application Agent

Autonomous form submission with human approval gates

RESPONSIBILITIES

Submits job applications via browser automation. Fills multi-page ATS forms, uploads documents, handles custom fields, and captures submission confirmation.

OUTPUTS

Submission confirmation (screenshot + reference ID), application status record, time-stamp

DECISION LOGIC

ALWAYS requires explicit human approval before submission. Never submits autonomously without approved package.

FAILURE HANDLING

On form error: capture screenshot and error text, escalate immediately. Never retry submission without human review — double submission risk. Dead-letter queue for failed submissions.

INPUTS

Approved application package (tailored resume, cover letter), application URL, form field mappings, user credentials vault reference

TOOLS

Playwright, form field detector, credential vault (HashiCorp Vault), screenshot capturer

06

Question Answering Agent

Intelligent response to application screening questions

RESPONSIBILITIES

Answers free-text and multiple-choice questions in ATS application forms. Retrieves supporting evidence from portfolio and resume. Computes confidence score per answer.

OUTPUTS

Answer text with confidence score (0-100), supporting evidence references, flagged concerns

DECISION LOGIC

Confidence $\geq 80\%$: auto-populate. 60-79%: surface to human for approval. $<60\%$: mandatory human input. Salary questions: always escalate to human.

FAILURE HANDLING

If evidence retrieval fails: flag question as unresolvable and route to human. Never fabricate — unanswerable questions get blank + human flag.

INPUTS

Question text, question type, context (JD + resume), portfolio evidence library

TOOLS

LLM (GPT-4o), semantic search (pgvector), evidence retriever, RAG pipeline

07

Interview Coach Agent

Preparation materials for every interview opportunity

RESPONSIBILITIES

Generates comprehensive interview preparation packs upon interview invitation. Includes predicted questions, STAR-format answers using portfolio evidence, company research, and key talking points.

OUTPUTS

Interview prep pack (PDF): Q&A, STAR examples, company brief, talking points, questions to ask

INPUTS

Job description, company name, interview type (technical/behavioural/panel), portfolio evidence

TOOLS

LLM (Claude 3.5), web researcher, company news scraper, PDF generator (WeasyPrint)

DECISION LOGIC + FAILURE

Triggered automatically on interview confirmation. On insufficient company data: use public information only and flag gaps. Output always reviewed by human before use.

08

Email Agent

Email monitoring, triage, and draft generation

RESPONSIBILITIES

Monitors inbox for recruiter communications, application status updates, and interview invitations. Classifies emails, extracts structured data, and drafts appropriate responses.

OUTPUTS

Classified email (intent, urgency, action required), draft response, extracted entities (date, company, role)

INPUTS

Email thread, application context, user tone preferences, previous correspondence

TOOLS

Gmail/Outlook API, LLM (GPT-4o), NER extractor, classification model

DECISION LOGIC + FAILURE

Urgency > HIGH: immediate notification. Never sends email without human approval. On API failure: queue emails for manual review with original content preserved.

Intelligence & Support Agents (9-15)

09

Calendar Agent

Automated interview scheduling and availability management

RESPONSIBILITIES

Manages interview scheduling by checking availability, proposing times, confirming bookings, and adding calendar events with prep reminders and meeting links.

OUTPUTS

Proposed time slots, confirmed event with meeting link, 24hr and 1hr reminders, prep pack trigger

INPUTS

Calendar availability, interview request details, timezone, buffer time preferences

TOOLS

Google Calendar API, Outlook Calendar API, timezone resolver, conflict detector

DECISION LOGIC + FAILURE

Only confirms bookings after user reviews proposed times. Never double-books. On calendar API failure: email candidate with manual scheduling link.

10

Recruiter Intelligence Agent

Company and recruiter profiling for strategic engagement

RESPONSIBILITIES

Builds intelligence dossiers on recruiters and hiring companies. Researches culture, hiring patterns, tech stack, recent news, and decision-maker profiles to inform strategic application targeting.

OUTPUTS

Intelligence report: culture signals, tech stack, average tenure, recent news, engagement strategy recommendation

INPUTS

Company name, recruiter name, LinkedIn URL, job listing metadata, industry sector

TOOLS

LinkedIn scraper, Glassdoor API, web search, news aggregator, company database

DECISION LOGIC + FAILURE

Minimum 3 independent data sources required for a reliable dossier. On insufficient data: partial report with confidence rating. Never publishes speculation as fact.

11

ATS Optimization Agent

Keyword extraction, scoring and gap analysis

RESPONSIBILITIES

Performs deep ATS analysis: extracts keywords from JD, computes TF-IDF importance weights, compares against resume embeddings, identifies gaps, and generates specific optimisation recommendations.

OUTPUTS

ATS score (0-100), keyword match matrix, gap list with priority, specific insertion recommendations

INPUTS

Resume text, job description text, ATS system type (Workday/Greenhouse/Lever), scoring weights

TOOLS

NLP pipeline (spaCy), TF-IDF scorer, embedding model (text-embedding-3-large), semantic matcher

DECISION LOGIC + FAILURE

Score $\geq$ 85: green light. 75-84: optimise further with provided recommendations. $<$ 75: block application, escalate with full gap analysis.

12

Learning Agent

Outcome-driven strategy improvement via reinforcement

RESPONSIBILITIES

Continuously analyzes application outcomes (responses, interviews, offers, rejections) to identify patterns and update strategies. Adjusts keyword weights, application targeting, and timing parameters.

OUTPUTS

Updated strategy parameters, keyword weight adjustments, targeting recommendations, performance trend report

INPUTS

Application history, outcome labels, ATS scores, interview results, rejection reasons (where available)

TOOLS

Statistical analysis, embedding comparison, A/B strategy tracker, recommendation engine

DECISION LOGIC + FAILURE

Minimum 10 outcomes required before making strategy updates. All changes are proposed to human for review — never autonomously modifies core profile data. Maintains version history of all strategy states.

13

Compliance Agent

Zero-fabrication enforcement and PII protection

RESPONSIBILITIES

Validates all generated content against the evidence library. Detects hallucinations, unsupported claims, PII leakage, and policy violations. Acts as the last gate before any external communication.

OUTPUTS

Compliance report: PASS/FAIL, violation list with severity, suggested corrections, evidence citations

INPUTS

Generated content (resume, cover letter, answers), evidence library, compliance rules, PII patterns

TOOLS

Hallucination detector, PII scanner (presidio), fact-checker, evidence retriever

DECISION LOGIC + FAILURE

Any CRITICAL violation: hard block — content never leaves system until remediated. HIGH violations: flag and escalate. Compliance check logs are append-only and immutable. Cannot be bypassed.

14

Analytics Agent

Real-time metrics, funnel analysis, and performance insights

RESPONSIBILITIES

Continuously aggregates application data into dashboards and reports. Tracks funnel conversion, ATS pass rates, agent performance, cost metrics, and time-to-outcome.

OUTPUTS

Live dashboard metrics, weekly performance reports, anomaly alerts, funnel conversion rates

INPUTS

Application records, outcome events, agent execution logs, timestamp data, cost ledger

TOOLS

Grafana API, PostgreSQL aggregation queries, Langfuse traces, alerting engine

DECISION LOGIC + FAILURE

Anomalies (sudden drop in ATS scores, zero applications in 48hrs) trigger immediate alerts. On data pipeline failure: uses last-known-good snapshot with staleness flag.

15

Supervisor Agent

Goal management and orchestration coordinator

RESPONSIBILITIES

The highest-level orchestrator. Receives user goals, decomposes them into sub-goals, assigns tasks to the Planner Agent, monitors overall progress, resolves conflicts between agents, and enforces system-wide policies.

OUTPUTS

Goal decomposition plan, task assignments, priority queue update, escalation decisions, system health report

INPUTS

User goals (job type, criteria, urgency), system state snapshot, active task queue, policy configuration

TOOLS

LangGraph state machine, task queue manager, conflict resolver, health monitor

DECISION LOGIC + FAILURE

Conflict resolution: priority-based with human override capability. On critical system failure: safe-mode activation — freeze all submissions, alert user, preserve state. Never loses task context on restart.

16

Planner Agent

Step-by-step execution planning per job opportunity

RESPONSIBILITIES

Creates detailed, sequenced execution plans for each job opportunity. Determines which agents to invoke, in what order, with what parameters, and with what conditional branches based on outcomes.

OUTPUTS

Execution DAG (directed acyclic graph), agent invocation sequence, decision points, estimated timeline

INPUTS

Job opportunity metadata, user constraints (time, budget, energy), agent capability registry, historical success patterns

TOOLS

LangGraph DAG builder, agent registry, historical pattern retriever, timeline estimator

DECISION LOGIC + FAILURE

Plan validation: check for circular dependencies, resource conflicts, and constraint violations before execution. On plan failure: incremental re-planning from last successful checkpoint.

System & Infrastructure Agents (17-20)

17

Verifier Agent

Output validation and accuracy verification

RESPONSIBILITIES

Independent validator that checks all agent outputs against the source evidence. Verifies factual accuracy, logical consistency, formatting integrity, and completeness before downstream processing.

OUTPUTS

Verification report with pass/fail per check, confidence score, specific violations, evidence citations

DECISION LOGIC + FAILURE

Independent of the originating agent — uses separate LLM call for verification (prevents self-hallucination). On verification failure: block output, log discrepancy, route to Compliance Agent for secondary review.

INPUTS

Generated content, source evidence library, validation rules, previous version (for diff)

TOOLS

Semantic similarity scorer, fact extractor, evidence retriever, diff engine

18

Memory Agent

Unified memory management across all agents

RESPONSIBILITIES

Manages the multi-tier memory hierarchy: working memory (current task context), session memory, long-term semantic memory (PostgreSQL + pgvector), episodic memory (application history), and procedural memory (learned strategies).

OUTPUTS

Relevant context window, memory update confirmations, decay/consolidation reports

DECISION LOGIC + FAILURE

Memory consolidation runs nightly — moves short-term working memories to long-term store via embedding compression. On retrieval failure: fallback to keyword search. Memory is never deleted, only deprecated with versioning.

INPUTS

Agent interactions, query context, memory type, recency/relevance weights, retrieval request

TOOLS

pgvector similarity search, Redis session cache, memory consolidation scheduler, embedding updater

19

Reflection Agent

Periodic meta-analysis for systemic improvement

RESPONSIBILITIES

Performs periodic (weekly) reflective analysis of the entire system's performance. Identifies blind spots, strategy stagnation, bias patterns, and improvement opportunities that individual agent metrics miss.

OUTPUTS

Reflection report: top 3 improvement recommendations, strategy hypotheses, A/B test proposals

INPUTS

Application history (30-day window), agent performance logs, outcome outcomes, strategy version history

TOOLS

Statistical analyzer, LLM (Claude 3.5 for reasoning), pattern detector, hypothesis generator

DECISION LOGIC + FAILURE

Triggered weekly or on significant outcome drop (>20% decline in any KPI). All recommendations require human approval before implementation. Reflection reports are preserved in episodic memory for meta-learning.

20

Error Recovery Agent

Intelligent failure handling and system resilience

RESPONSIBILITIES

Monitors all agent executions for failures, classifies error types (transient/permanent/partial), selects and executes the appropriate recovery strategy, and escalates to human when automated recovery is not possible.

OUTPUTS

Recovery action (retry/fallback/escalate), incident report, circuit breaker state update, postmortem trigger

INPUTS

Error context (agent ID, error type, stack trace, input state), retry history, circuit breaker state

TOOLS

Circuit breaker (Resilience4j pattern), retry scheduler, dead-letter queue manager, alerting system

DECISION LOGIC + FAILURE

Transient errors: exponential backoff (1s, 2s, 4s, 8s, 16s). Permanent errors: immediate escalation + task archival. Partial failures: complete safe portions, human input for remainder. Circuit breaker opens after 5 consecutive failures within 60s.

Multi-Agent Orchestration

The orchestration layer is the nervous system of the platform. It coordinates the 20 agents through a **Supervisor** → **Planner** → **Executor** → **Reviewer** → **Verifier** hierarchy, using LangGraph to define the execution DAG and Redis BullMQ as the asynchronous task bus. The design supports parallel execution of independent tasks, conditional branching on outcomes, and deterministic human escalation at defined quality gates.

Orchestration Flow Diagram

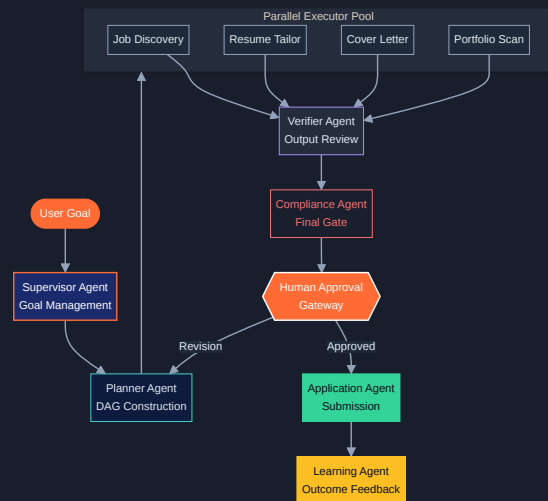


Figure 5 — Orchestration Architecture: Supervisor to Learning feedback loop with Human Approval Gateway.

Communication Patterns

PATTERN	USED FOR	TECHNOLOGY
Request/Reply	Synchronous agent calls	FastAPI REST
Pub/Sub Events	Outcome notifications	Redis Pub/Sub
Task Queue	Async agent work units	BullMQ (Redis)
Streaming	LLM output tokens	SSE / WebSocket
Broadcast	System state updates	LangGraph state

Coordination Protocol

Task Lifecycle

Tasks are published to typed queues (job-discovery, resume-tailor, etc.) by the Planner. Agents consume tasks, update shared state via the LangGraph StateGraph, and publish completion events. Downstream agents subscribe to relevant completion events to trigger their own execution.

Conflict Resolution

The Supervisor Agent maintains a global goal registry. When two agents produce conflicting outputs (e.g., different ATS scores for the same resume), the Supervisor invokes a tiebreaker: prefer the higher-confidence result, flag the discrepancy, and log both for audit. Persistent conflicts escalate to human.

Human-in-the-Loop Gates

Four mandatory escalation points: (1) Job approval — user confirms which discovered jobs to pursue. (2) Application approval — user approves tailored resume + cover letter bundle before submission. (3) Low-confidence Q&A — user provides answers to questions below threshold. (4) Strategy changes — user approves Learning Agent recommendations.

Parallel Execution Model

Once a job is approved for application, the Portfolio Agent, ATS Optimization Agent, and Recruiter Intelligence Agent execute in parallel. Resume Tailoring and Cover Letter generation are serialized (Cover Letter depends on tailored resume). This hybrid model achieves maximum throughput while respecting data dependencies.

State Management

LangGraph StateGraph maintains a typed state object per job application instance. State includes: current stage, all agent outputs, human decisions, error counts, and audit trail. State is persisted to PostgreSQL with every transition — crash recovery always resumes from the last committed state checkpoint.

Sequence Diagrams

Sequence diagrams illuminate the temporal interactions between system components for key workflows. Each diagram shows the exact message flow, decision points, and failure paths for critical platform capabilities.

Sequence 1 — Job Discovery and Ranking Flow

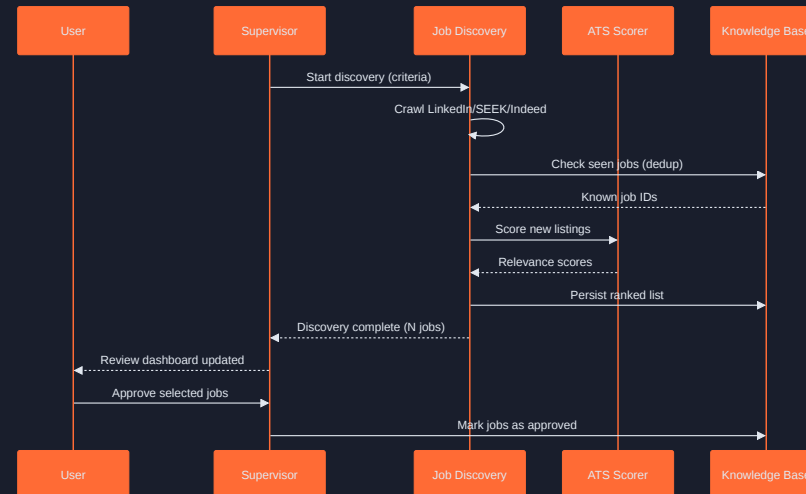
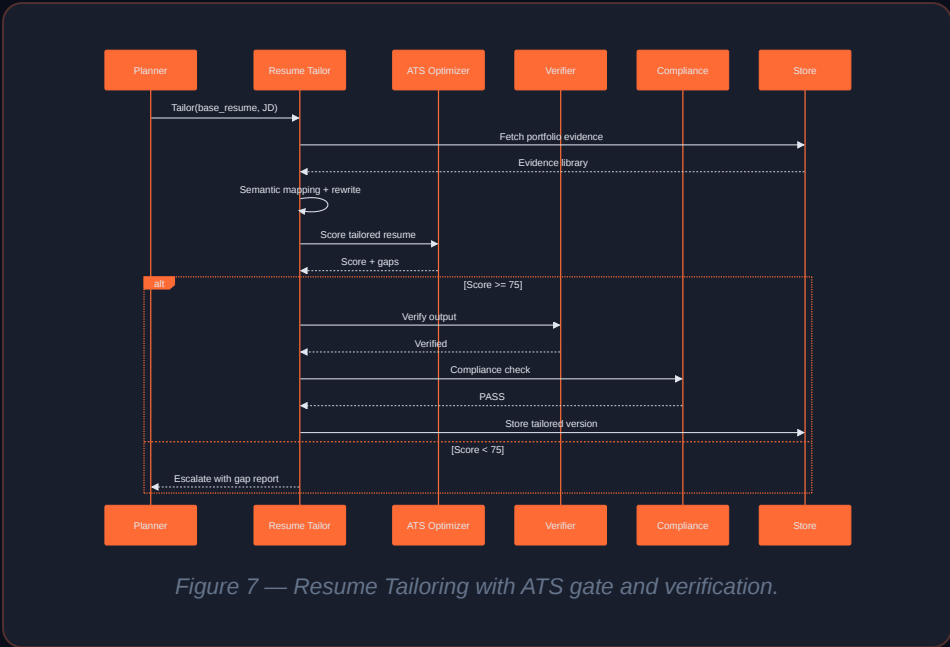
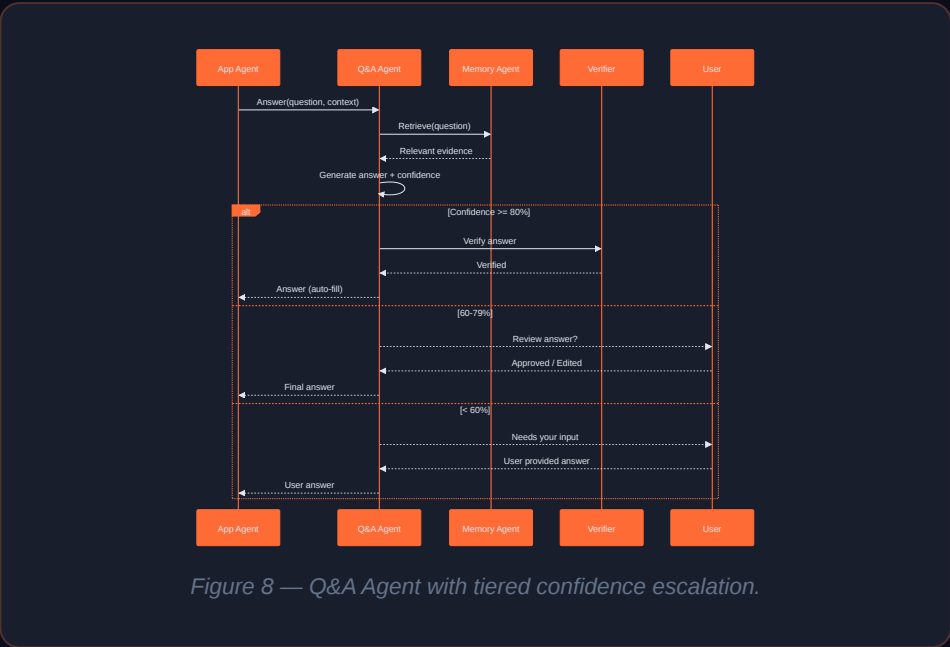


Figure 6 — Job Discovery to Human Approval sequence.

Sequence 2 — Resume Tailoring Flow



Sequence 3 — Question Answering Flow



Sequence 4 — Full Application Lifecycle

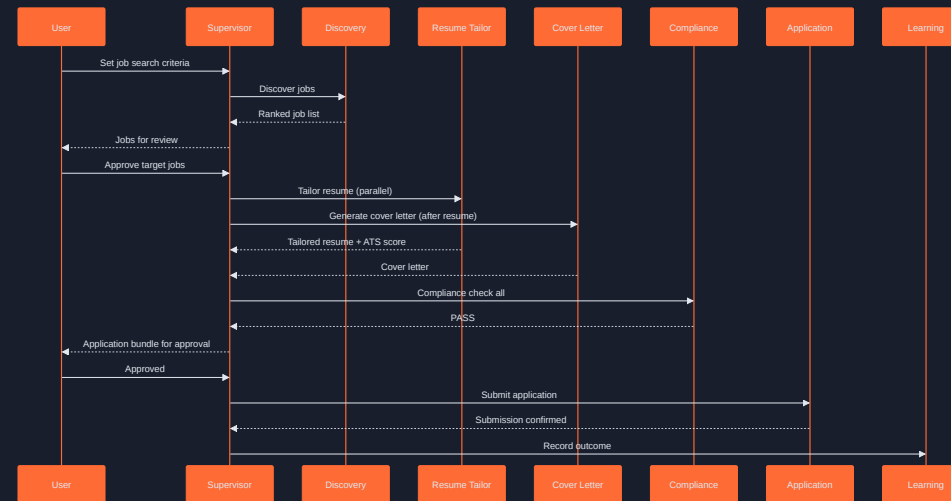


Figure 9 — Full Application Lifecycle from discovery through to submission and learning feedback.

Sequence 5 — Continuous Learning Loop

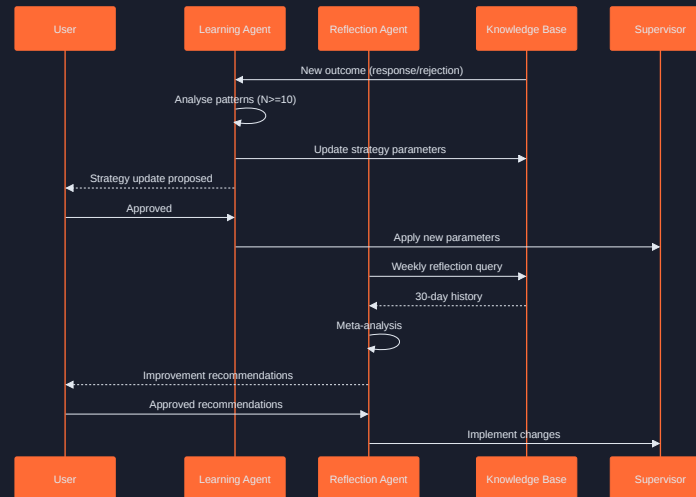


Figure 10 — Continuous Learning Loop: Learning Agent and Reflection Agent improve strategy over time with human oversight.

Resume Tailoring Pipeline

The resume tailoring pipeline is the most critical and technically demanding component of the platform. It must produce ATS-optimised, evidence-backed resumes that match the job description — while preserving the source document's formatting with pixel-perfect fidelity. Typography, column layout, spacing, font sizes, and margins must remain identical to the original. Only content changes are permitted.

Format Preservation Constraint (Non-Negotiable): The tailoring pipeline operates at the content layer only. It never modifies the structural XML of the DOCX file, never changes CSS styles, never adjusts paragraph spacing, never alters font selections, and never shifts page margins. Vikram's resume uses a two-column asymmetric layout with coral/orange accent icons — this design must be preserved verbatim in every tailored version.

Pipeline Architecture Diagram



Syntax error in text
mermaid version 10.9.6

Figure 11 — Resume Tailoring Pipeline: 12-stage processing from input to verified tailored output.

Pipeline Stage Details

STAGE	PROCESS	OUTPUT
-------	---------	--------

1. PDF/DOCX Parsing	Extract structured content using PyMuPDF and pdfplumber. Preserve paragraph structure, style references, and layout zones.	Structured document model with style map
2. Portfolio Enrichment	Retrieve current evidence from portfolio library — metrics, projects, achievements — to supplement resume facts.	Enriched evidence context
3. JD Analysis	Extract required skills, experience, keywords, responsibilities, and culture signals using NLP pipeline (spaCy + LLM).	Weighted keyword list, JD summary
4. Skill Mapping	Map JD requirements against candidate's skill inventory using semantic similarity. Identify primary matches, secondary matches, and gaps.	Match matrix with confidence scores
5. Semantic Matching	Compute embedding similarity between JD requirements and resume sections. Score each section's relevance to the role.	Section relevance scores, reorder recommendations
6. Achievement Rewriting	Reframe existing achievements using JD language. Emphasise most-relevant metrics. Use STAR format where possible.	Rewritten bullet points with evidence citations

STAGE	PROCESS	OUTPUT
7. Keyword Optimization	Insert missing high-priority keywords naturally into existing bullet points. Never keyword-stuff — must read naturally to a human reviewer.	Keyword-enhanced content with insertion log
8. Format Preservation Check	Verify that no structural changes have been introduced. Compare paragraph styles, font sizes, and layout dimensions against source template.	Format compliance report (PASS/FAIL)
9. Grammar and Tone Check	Proofread tailored content for grammatical consistency, active voice, appropriate professional tone, and tense uniformity (past tense for completed roles).	Corrected content with change log
10. ATS Validation	Run full ATS Optimization Engine scoring against the JD. Gate: score must meet threshold (default 75%) to proceed.	ATS score (0-100), pass/fail, gap list
11. Export	Render to PDF and DOCX using original template styles. Validate output files for corruption, page count, and visual integrity.	Tailored resume PDF + DOCX artifacts

12. Verification	Independent Verifier Agent cross-checks tailored content against evidence library. Compliance Agent performs final gate check.	Verification report, compliance certificate
---------------------	--	---

Critical Constraints

Zero Layout Mutation

The pipeline operates on the text content layer only. Column widths, margins (2cm left sidebar, ~70% main column), font families, and inter-paragraph spacing are read-only. Any proposed change that would affect layout is rejected at Stage 8 and triggers a rewrite from Stage 4.

Evidence-Only Rewriting

Every rewritten bullet point must be traceable to at least one evidence source: a GitHub repository, portfolio project entry, employment record, or certified metric. The Verifier Agent validates this traceability chain for every modified sentence.

Version Lineage

Every tailored version is stored with full lineage: base resume version, JD hash, evidence snapshot, keyword list used, ATS score, and operator decision log. This supports A/B analysis of which tailoring strategies produce the highest interview conversion rates.

ATS Optimization Engine

The ATS Optimization Engine is a multi-stage scoring and recommendation system that quantifies how well a tailored resume will perform against an Applicant Tracking System's automated screening. Unlike simple keyword counters, the engine uses semantic embeddings, experience scoring models, and machine-learned ATS-system-specific weights to produce an 0-100 confidence score.

Scoring Architecture



Syntax error in text
mermaid version 10.9.6

Figure 12 — ATS Engine internal scoring stages.

Score Composition

COMPONENT	WEIGHT	DESCRIPTION
Keyword Match	35%	Exact and stemmed keyword presence vs JD required terms
Semantic Similarity	25%	Embedding cosine similarity between resume sections and JD

Experience Match	20%	Years of experience, seniority level, industry match
Skill Coverage	12%	Required vs present skills, with must-have vs nice-to-have weighting
Achievement Quality	8%	Quantified metrics, action verbs, STAR format compliance

Keyword Extraction Algorithm

The keyword extraction process uses a three-stage pipeline applied to each job description:

- 1. TF-IDF Analysis:** Compute term frequency-inverse document frequency across a corpus of 10,000+ job descriptions to identify high-signal keywords. Common filler terms are filtered out; role-specific technical terms are upweighted.
- 2. Named Entity Recognition:** spaCy NER pipeline identifies technologies (PRODUCT), organizations (ORG), and skills (custom entity type). These receive higher priority in matching regardless of TF-IDF score.
- 3. Dependency Parsing:** Identify requirement phrases (e.g., "must have 5+ years of Python") to extract structured requirement slots: skill, level, duration, and importance qualifier (required/preferred).

Threshold Configuration

SCORE RANGE	STATUS	ACTION
90-100	Exceptional	Proceed immediately, flag as high-priority application
85-89	Strong	Proceed to submission workflow
75-84	Good	Apply further optimisations from recommendation list, then proceed

65-74	Marginal	Escalate to human — show gap analysis, request manual enhancement
Below 65	Insufficient	Block application. Return to Planner for strategy revision.

Skill Gap Detection

The skill gap detector classifies each JD requirement into one of four states:

- **Direct Match** — Exact keyword present in resume with evidence
- **Semantic Match** — Semantically equivalent skill (e.g., "LangChain" vs "LLM frameworks") — partial credit
- **Portfolio Match** — Skill demonstrated in portfolio but not explicitly stated in resume — recommendation to add
- **True Gap** — Skill absent from resume and portfolio — reported as gap with severity (critical/major/minor)

Portfolio Integration Advantage: Vikram's portfolio at forgotten-mistery.web.app contains verified evidence for LangChain, Langfuse, React, pgvector, and COBOL/Mainframe. The ATS Engine queries the evidence library before classifying any skill as a "True Gap" — this significantly reduces false-gap reporting compared to resume-only analysis.

Data Model & Entity Relationship Diagram

The platform uses PostgreSQL as its primary relational store, augmented with the pgvector extension for semantic similarity search. The data model supports the full application lifecycle, from user profile through to learning outcomes, with comprehensive audit trails at every stage.

Core Domain ERD

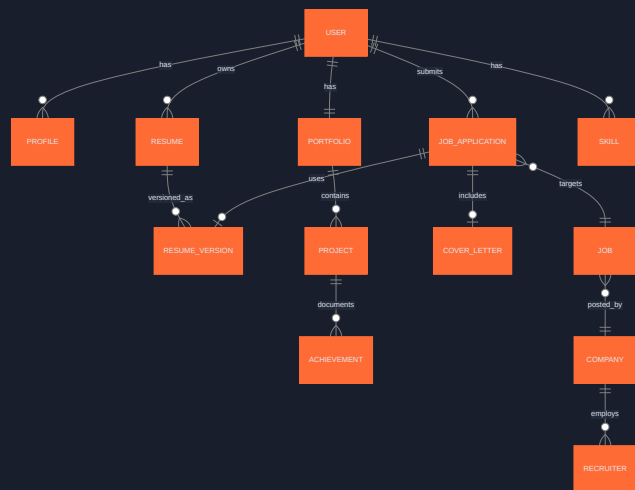


Figure 13 — Core Domain ERD: User and Application entities.

Agent and Audit ERD

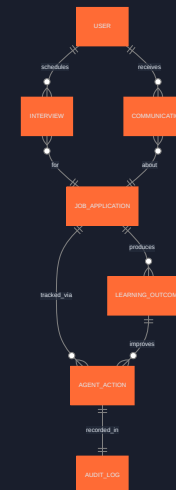


Figure 14 — Agent and Audit ERD: Action tracking and learning loop.

Key Database Design Decisions

resume_version	user_id, ats_score, created_at	Stores embedding vector for semantic search
job	company_id, posted_at, relevance_score	Full-text search on description column
job_application	user_id, status, submitted_at	Partitioned by month for analytics
agent_action	agent_id, job_application_id, timestamp	Immutable — no UPDATE/DELETE allowed
audit_log	entity_type, entity_id, created_at	Append-only, replicated to S3
learning_outcome	application_id, outcome_type, recorded_at	Triggers Learning Agent via event

Entity Descriptions

Core Entities

- **User** — Platform account, preferences, API credentials
- **Profile** — Professional profile, target roles, salary expectations
- **Resume** — Base resume document with style metadata
- **ResumeVersion** — JD-specific tailored version with ATS score and embedding vector
- **Portfolio** — Link to portfolio site, crawl status, last updated
- **Project** — Individual portfolio project with evidence data
- **Achievement** — Quantified metric linked to a project and employment record
- **Skill** — Tagged skill with proficiency level and evidence count

Application Entities

- **Job** — Scraped job listing with JD, URL, company, salary, posting date
- **JobApplication** — Application record linking user, job, resume version, and status
- **CoverLetter** — Generated letter with evidence citations and authenticity score
- **Company** — Company profile with tech stack, culture signals, recruiter contacts
- **Recruiter** — Individual recruiter with contact history and intelligence dossier
- **Interview** — Scheduled interview with type, date, prep pack, and outcome
- **Communication** — Email thread or message linked to an application

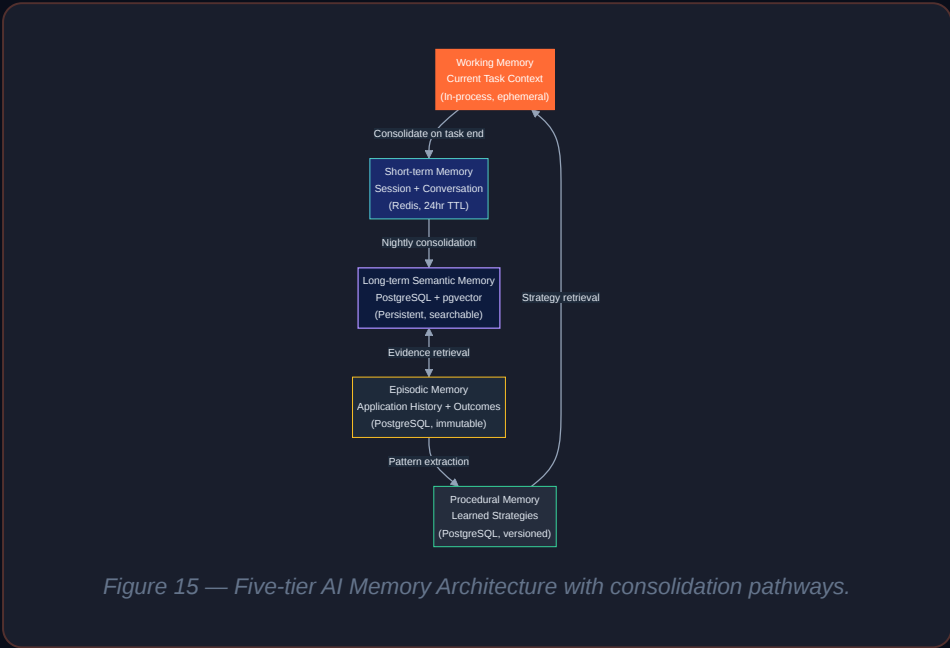
System Entities

- **AgentAction** — Immutable record of every agent invocation with inputs, outputs, and confidence
- **AuditLog** — Append-only compliance log for all data changes
- **LearningOutcome** — Outcome signal (interview/rejection/offer) linked to application for reinforcement learning
- **StrategyVersion** — Versioned application strategy parameters updated by Learning Agent
- **EvidenceCache** — Cached portfolio evidence with embedding vectors and staleness flags

AI Memory Architecture

The memory architecture provides the platform's cognitive continuity across sessions, agents, and time. Inspired by human memory research, it is structured into five distinct tiers: Working, Short-term, Long-term Semantic, Episodic, and Procedural. Each tier serves different temporal and functional needs, with the Memory Agent managing transitions between tiers and serving retrieval requests.

Memory Hierarchy Diagram



Memory Tier Specifications

TIER	STORAGE	CAPACITY	RETRIEVAL
Working	LangGraph state (in-process)	Current task context window (128k tokens)	Direct access, millisecond
Short-term	Redis cache	Last 50 interactions per session	Key lookup, sub-millisecond
Long-term Semantic	PostgreSQL + pgvector	All historical content as embeddings (1536-dim)	Cosine similarity search
Episodic	PostgreSQL (application_events)	All application history, outcomes, emails	SQL query + full-text search

Procedural	PostgreSQL (strategy_versions)	Versioned strategy parameters	Latest version read, history available
------------	-----------------------------------	-------------------------------------	--

Memory Consolidation Process

Nightly Consolidation

A scheduled process runs at 02:00 AEST nightly. It scans Redis for Short-term memories older than 6 hours, computes embeddings using text-embedding-3-large, and inserts them into the pgvector store with appropriate metadata tags (agent_id, timestamp, context_type). Redis entries are then expired.

Retrieval Augmented Context

When an agent begins a task, the Memory Agent performs a semantic search across all memory tiers using the task context as query. The top-K most relevant memories are injected into the agent's working context window, enabling continuity across sessions without manual re-prompting.

Knowledge Graph

The Knowledge Graph provides a connected representation of all entities in the platform — linking the user's identity through their skills, projects, technologies, achievements, companies, and job applications into a coherent, traversable graph. This enables evidence tracing (portfolio project → achievement → resume bullet point → application), skill inference, and strategic career path analysis.

Core Knowledge Graph

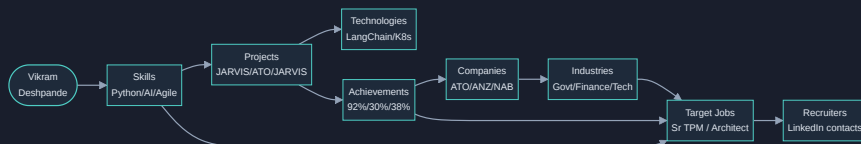


Figure 16 — Knowledge Graph: Entity interconnections from user identity to target roles.

Evidence Trace Example

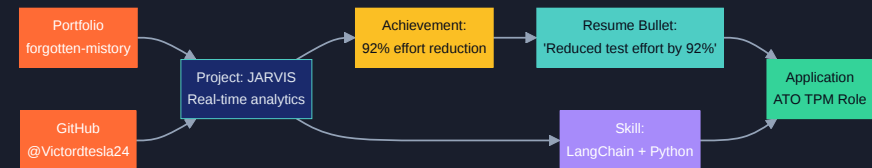


Figure 17 — Evidence Trace: From portfolio source through achievement to application submission.

Graph Query Examples

The knowledge graph enables powerful cross-entity queries that would be impossible with a flat relational model:

- **"Find all achievements linked to roles requiring Kubernetes"** — traverses Skills → Projects → Achievements → Returns quantified outcomes for container orchestration work

- **"Which companies in Vikram's target industries are currently hiring?"** — traverses Industries → Companies → Jobs → Returns filtered, ranked opportunities
- **"What evidence supports the claim of AI/ML expertise?"** — traverses Skills(AI/ML) → Projects → Achievements → Portfolio evidence → Returns verifiable proof chain
- **"Identify skill gaps for a Senior AI Architect role"** — compares required skills subgraph against current skills subgraph, returns delta with portfolio coverage indicators

Security Architecture

The platform handles sensitive personal and professional data including resumes, employment history, contact details, credentials, and application communications. Security is implemented using a Zero Trust model — every request is authenticated, every action is authorised, every data access is logged, and no component implicitly trusts any other. The architecture is designed for compliance with the **Australian Privacy Act 1988** and GDPR.

Authentication & Authorisation

OAuth2 + JWT Authentication

All user-facing APIs require a valid JWT Bearer token obtained via OAuth2 PKCE flow (NextAuth.js). Tokens expire after 1 hour. Refresh tokens are rotated on every use and stored in HttpOnly cookies. API-to-API communication between services uses short-lived machine tokens (15-minute TTL).

Role-Based Access Control (RBAC)

Three roles: **Owner** (full access, can modify profile and approve submissions), **Reviewer** (read-only access to applications and reports, cannot approve submissions), and **System** (internal service accounts with scoped permissions per agent type). Permissions are checked on every API call via middleware — never assumed.

Encryption

DATA LAYER	ENCRYPTION	KEY MANAGEMENT
In Transit	TLS 1.3 for all connections (API, DB, queue)	ACM certificates, auto-renewed
At Rest (DB)	AES-256 via RDS encryption	AWS KMS, customer-managed key
At Rest (S3)	SSE-KMS per object	AWS KMS, separate key per environment
PII Fields	Field-level encryption (AES-256-GCM)	Application-layer key, rotated quarterly
Credentials	HashiCorp Vault (AES-256)	Vault unsealing via AWS KMS

PII Handling & Data Privacy

PII Identification and Masking

Microsoft Presidio scans all outbound content (emails, cover letters, Q&A answers) for PII patterns: phone numbers, email addresses, passport numbers, tax file numbers (TFN), and bank accounts. Detected PII is masked in logs, replaced with tokens in audit trails, and only rendered in the approved UI context.

Data Retention and Right to Erasure

Application data is retained for 24 months by default. Users can request full data export or deletion via the dashboard. Deletion anonymises the record (replaces PII with hashed identifiers) rather than hard-deleting, preserving aggregate analytics while removing personal data. Audit logs retain anonymised event records indefinitely.

Threat Model Summary

THREAT	MITIGATION
Credential theft	HashiCorp Vault, no secrets in env vars or code, Vault agent injection

Prompt injection	Input sanitisation, templated prompts, output validation by Compliance Agent
Data exfiltration	Egress filtering, PII masking, audit logging of all external API calls
Unauthorized submission	Mandatory human approval gate, RBAC on submit action, immutable audit log
Bot detection by job sites	Playwright stealth mode, rate limiting, randomised delays, ethical crawling headers
LLM hallucination	Compliance Agent, Verifier Agent, evidence-only rewriting constraint

Zero Trust Principles

- **Verify explicitly** — All requests authenticated via JWT; no implicit trust between services
- **Least privilege access** — Agent service accounts scoped to minimum required permissions
- **Assume breach** — All actions logged; alert on anomalous patterns; blast radius minimised through namespace isolation
- **Micro-segmentation** — Kubernetes NetworkPolicies restrict pod-to-pod communication to declared routes only

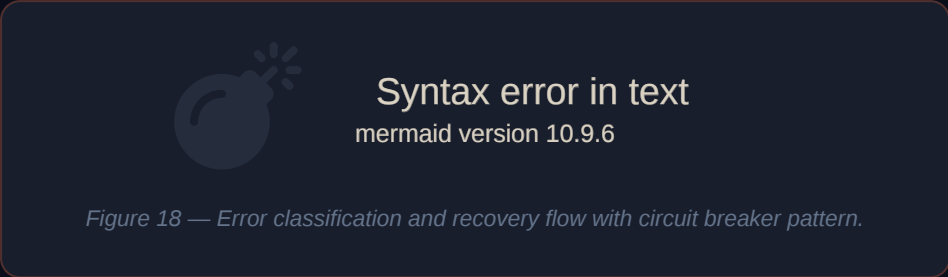
Error Handling & Recovery

Failure Taxonomy

TYPE	DEFINITION	EXAMPLES	STRATEGY
Transient	Temporary failure likely to self-resolve	Network timeout, API rate limit, LLM latency spike	Exponential backoff with jitter (3-5 retries)
Permanent	Failure that cannot resolve without intervention	Invalid credentials, missing required data, ATS score below threshold	Immediate escalation + task archival
Partial	Some subtasks succeeded, others failed	Resume tailored but cover letter failed; 3 of 5 form fields submitted	Complete safe portions; human input for remainder
Data	Input or output data quality failure	Corrupt PDF, empty JD, hallucinated content detected	Reject input, notify, request clean input

Security	Policy or compliance violation	PII detected in output, fabricated claim found	Hard block, alert, audit log, never retry without human review
-----------------	--------------------------------	--	--

Retry Strategy



Circuit Breaker Configuration

AGENT/SERVICE	OPEN THRESHOLD	HALF-OPEN AFTER	CLOSE THRESHOLD
Job Discovery	5 failures in 60s	30s	2 successes
Resume Tailor	3 failures in 120s	60s	1 success
Application Submit	1 failure	Manual reset	Human approval
LLM API (OpenAI)	5 failures in 30s	15s (try Claude fallback)	2 successes
Email Agent	3 failures in 60s	120s	1 success

Human Escalation Triggers

Mandatory Escalation Events

- ATS score below 65% after 2 optimisation attempts
- Compliance Agent CRITICAL violation on any output
- Application submission failure (any form error)
- Q&A confidence below 60% for any required field
- Circuit breaker open on Application Submit agent
- Security anomaly detected (unusual access pattern)
- LLM hallucination detected by Verifier Agent
- Salary range question in any application form

Dead Letter Queue (DLQ) Management

Failed tasks that exhaust their retry budget are moved to the DLQ in Redis. Each DLQ entry contains: original task payload, error history, attempt count, and classification. The DLQ dashboard surfaces these to the user with recommended actions. Tasks expire from the DLQ after 7 days if not acted upon, with a final alert sent 24 hours before expiry.

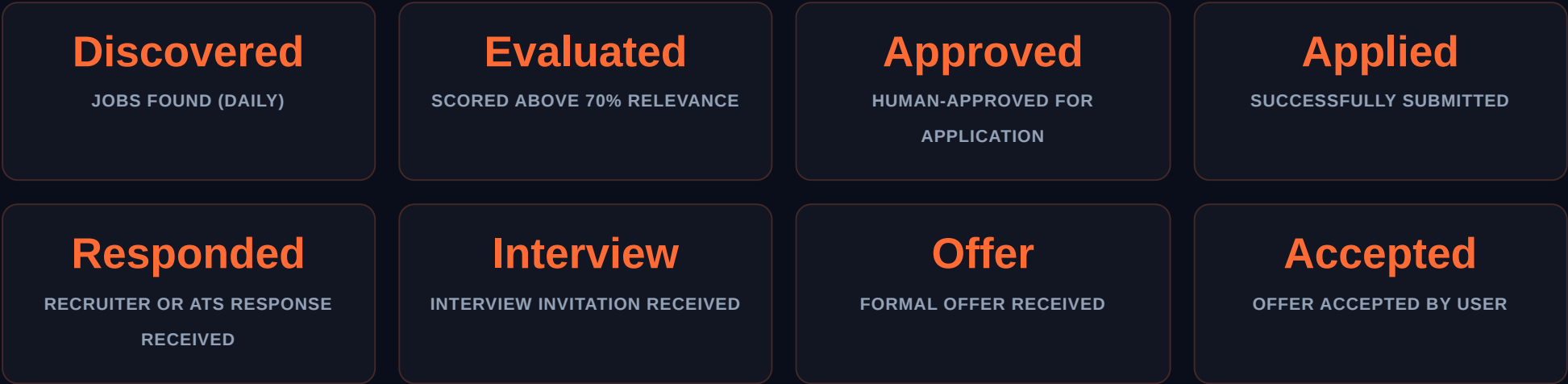
Recovery Workflows

- **Format Corruption Recovery:** Revert to base resume, re-run tailoring from Stage 4 with stricter format constraints. Log the violation pattern for ATS Engine improvement.
- **LLM Provider Outage:** Automatic failover — OpenAI GPT-4o → Claude 3.5 Sonnet → Gemini Pro. All models produce identical output contracts via standardised prompts.
- **Database Connection Lost:** Write queue buffers mutations. On reconnection, replay mutations in order. Read requests serve from Redis cache with staleness flag displayed to user.

Analytics & KPIs

The Analytics Engine aggregates every platform event — from job discovery through to offer outcomes — into a real-time dashboard and weekly performance reports. Metrics are tracked at three levels: funnel conversion, agent performance, and strategic effectiveness. The Analytics Agent (Agent 14) maintains these metrics and triggers alerts on anomalous patterns.

Application Funnel Metrics



Key Performance Indicators

KPI	TARGET	ALERT THRESHOLD	MEASUREMENT
-----	--------	-----------------	-------------

Applications per day	5-10	Below 2 for 48hrs	Count of submitted applications
ATS pass rate	85%+	Below 70%	% submissions with ATS score >= 75
Interview conversion	8-12%	Below 4%	Interviews / Applications (rolling 30 days)
Time-to-apply	< 10 min	> 30 min	Approval timestamp - Job discovery timestamp
Resume effectiveness	> 80%	Below 65%	Composite ATS score across all tailored versions
Compliance pass rate	100%	Any FAIL	% outputs passing Compliance Agent gate
Agent success rate	> 95%	Below 90%	Successful agent executions / Total invocations
Cost per application	< A\$0.50	> A\$2.00	LLM API costs + infra / Applications submitted

Agent Performance Metrics

AGENT	PRIMARY METRIC	SECONDARY METRIC
Job Discovery	Jobs found per crawl	Deduplication rate
Resume Tailor	Average ATS score delta	Format violation rate
Cover Letter	Authenticity score	Human edit rate
ATS Optimizer	Score accuracy vs outcome	Keyword gap recall
Compliance	False positive rate	Processing latency
Learning	Strategy improvement rate	Recommendation acceptance rate
Error Recovery	Recovery success rate	Time-to-recovery

Reporting Schedule

Real-time Dashboard (Grafana)

Live counters for applications in flight, agent health, queue depth, and ATS scores. Updated every 30 seconds. Accessible via Next.js dashboard with role-based visibility.

Weekly Performance Report (Auto-generated PDF)

Generated every Monday at 07:00 AEST. Includes: applications submitted, funnel conversion rates, top-performing resume versions, keyword trends, and strategic recommendations from the Reflection Agent. Delivered by email and available in dashboard.

Technology Stack Recommendations

Every technology choice in this stack was selected for three criteria: (1) alignment with the platform's multi-agent AI architecture, (2) demonstrated reliability at production scale, and (3) direct leverage of Vikram's existing expertise — specifically LangChain, Langfuse, Python, TypeScript, React, PostgreSQL, Kubernetes, and cloud-native tooling already validated in the JARVIS and ATO Evidence Harness projects.

LAYER	TECHNOLOGY	RATIONALE
Frontend	Next.js 14 + TypeScript + Tailwind + shadcn/ui	SSR for dashboard performance; App Router for complex layouts; Tailwind for rapid consistent styling matching existing portfolio design
Backend API	FastAPI (Python) + Node.js (TypeScript)	FastAPI for AI/ML agent orchestration (native Python LangChain integration); Node.js for real-time WebSocket events and email webhooks
Agent Framework	LangGraph + LangChain	LangGraph provides deterministic DAG execution with state persistence — critical for multi- agent coordination and crash recovery. Matches existing Vikram expertise.

Primary Database	PostgreSQL + pgvector	PostgreSQL proven at Vikram's ANZ telemetry scale (10k+ concurrent). pgvector enables semantic memory search without a separate vector database for most use cases.
Vector Store	Pinecone (primary) / Weaviate (fallback)	Pinecone for high-performance semantic search at scale; Weaviate as self-hosted fallback for cost optimisation in development environments
Queue / Cache	Redis + BullMQ	Sub-millisecond latency for session cache; BullMQ provides typed job queues with retry/DLQ built-in. Industry standard for Node.js agent task buses.
Search	Elasticsearch	Full-text search across job descriptions, application history, and portfolio content. Faceted filtering for the job discovery dashboard.
Auth	NextAuth.js + OAuth2	Handles Google, LinkedIn, and Microsoft social login out of the box. PKCE flow for secure token exchange. Integrates natively with Next.js.
Monitoring	Langfuse + Grafana + Prometheus	Langfuse provides LLM trace visibility, hallucination scoring, and prompt versioning — Vikram already uses this in production. Grafana for system metrics. Matches existing toolchain.

LAYER	TECHNOLOGY	RATIONALE
AI Models (LLM)	GPT-4o (primary), Claude 3.5 Sonnet (creative), Gemini Pro (fallback)	GPT-4o for structured output and JSON mode; Claude 3.5 for long-form cover letters with nuanced tone; Gemini Pro as cost-optimised fallback
Embeddings	text-embedding-3-large (OpenAI)	1536-dimensional embeddings with state-of-the-art semantic retrieval accuracy. Essential for portfolio evidence matching and ATS semantic scoring.
Browser Automation	Playwright (Python + Node.js)	Cross-browser support for all major ATS systems (Workday, Greenhouse, Lever, SmartRecruiters). Stealth mode for bot detection avoidance. Native async support.
Document Generation	python-docx + WeasyPrint	python-docx for content-layer DOCX manipulation (format preservation); WeasyPrint for pixel-perfect PDF generation from HTML templates
Resume Parsing	PyMuPDF + pdfplumber	PyMuPDF for fast text extraction with position data; pdfplumber for table and column detection in multi-column resume layouts

Cloud	AWS (EKS, RDS, S3, SQS, KMS)	AWS ap-southeast-2 (Sydney) for Australian data residency compliance. EKS for Kubernetes orchestration. RDS for managed PostgreSQL with Multi-AZ failover.
CI/CD	GitHub Actions	Native integration with GitHub (@Victordtesla24). Matrix testing across Python versions. Docker image build + ECR push + EKS rolling deploy in one workflow.
IaC	Terraform + Pulumi	Terraform for stable AWS infrastructure (VPC, RDS, EKS); Pulumi for dynamic resource provisioning in agent worker scaling policies
Secret Management	HashiCorp Vault	Vault agent injection for Kubernetes pods — no secrets in environment variables or container images. Dynamic secrets for database credentials with 1-hour TTL.

Implementation Roadmap

The 16-week implementation roadmap is structured as four sequential phases, each delivering a vertical slice of production-ready capability. Each phase produces a shippable increment — Phase 1 alone is useful without Phase 2. Dependencies are explicit; phases do not begin until the prior phase's acceptance criteria are verified.

Phase 1 (Weeks 1-4): Foundation

Goal: Core infrastructure, database, authentication, and MVP resume pipeline.

- AWS EKS cluster provisioning (Terraform)
- PostgreSQL RDS + pgvector extension deployment
- Redis cluster for caching and queues
- NextAuth.js authentication with OAuth2 (Google + LinkedIn)
- Resume upload and PDF parsing (PyMuPDF + pdfplumber)
- Basic resume tailoring pipeline (Stages 1-7)
- Portfolio scraper MVP (forgotten-mistory.web.app)
- Dashboard skeleton with application tracker
- Langfuse integration and basic monitoring

Phase 2 (Weeks 5-8): Intelligence

Goal: Full agent architecture, ATS Engine, and cover letter automation.

- LangGraph multi-agent framework deployment
- Supervisor + Planner + Executor agent framework
- ATS Optimization Engine (full scoring pipeline)
- Cover Letter Agent with company research integration
- Job Discovery Agent (LinkedIn + SEEK + Indeed crawlers)
- Question Answering Agent with RAG pipeline
- Compliance Agent with hallucination detection
- Human approval workflow UI
- Vector store integration (Pinecone)

Phase 3 (Weeks 9-12): Automation

Goal: End-to-end application submission, email, calendar, and interview prep.

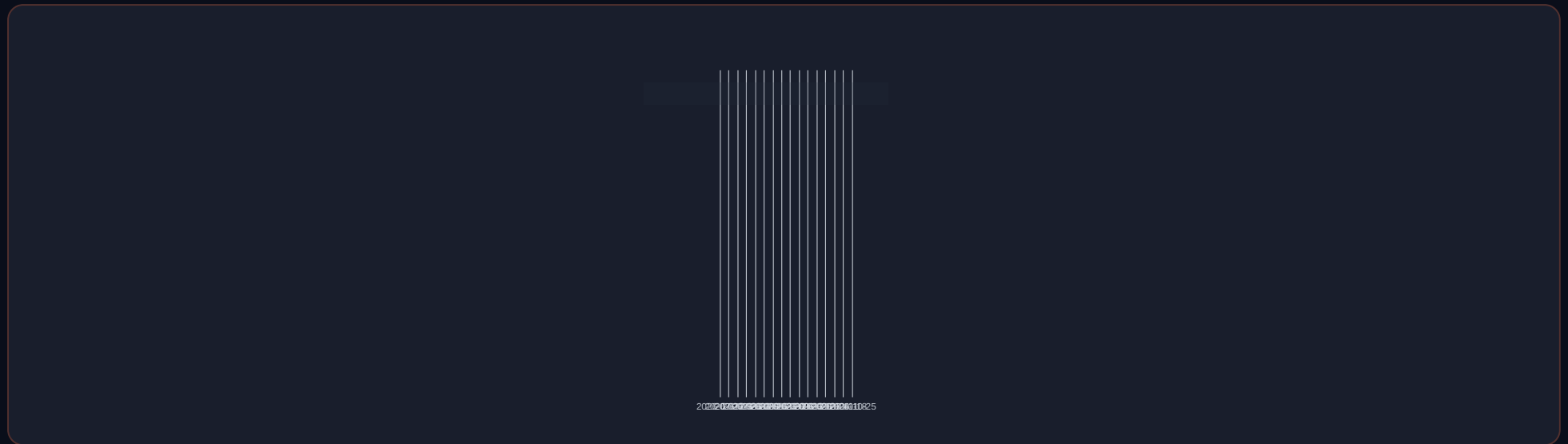
- Application Agent (Playwright browser automation)
- ATS form detection and field mapping
- Email Agent with Gmail/Outlook integration
- Calendar Agent with Google Calendar + O365
- Interview Coach Agent with prep pack generation
- Recruiter Intelligence Agent
- HashiCorp Vault credential management
- Full audit logging and compliance reporting
- Production security hardening

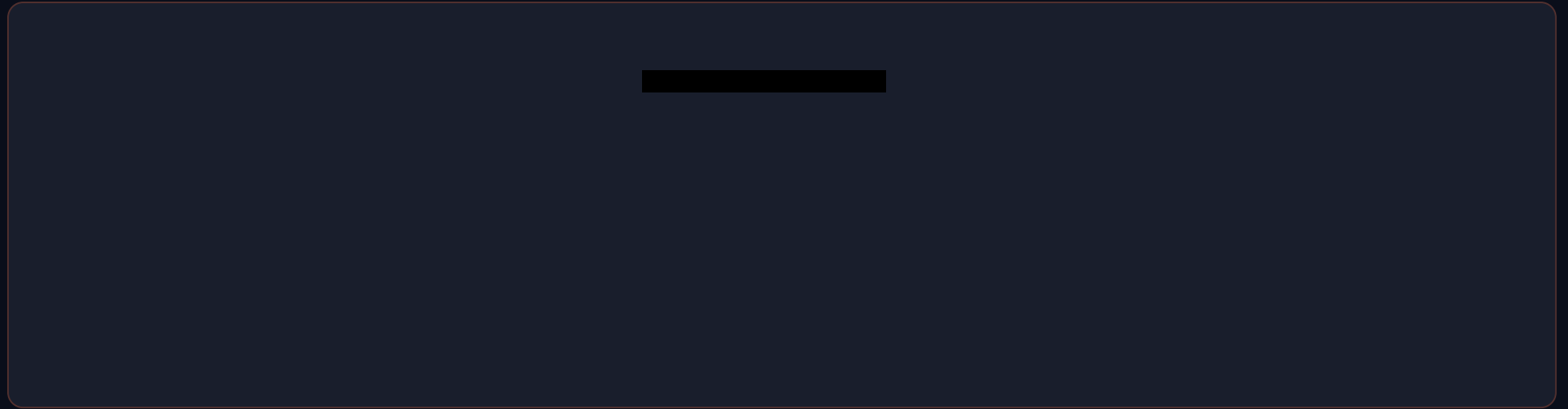
Phase 4 (Weeks 13-16): Learning & Scale

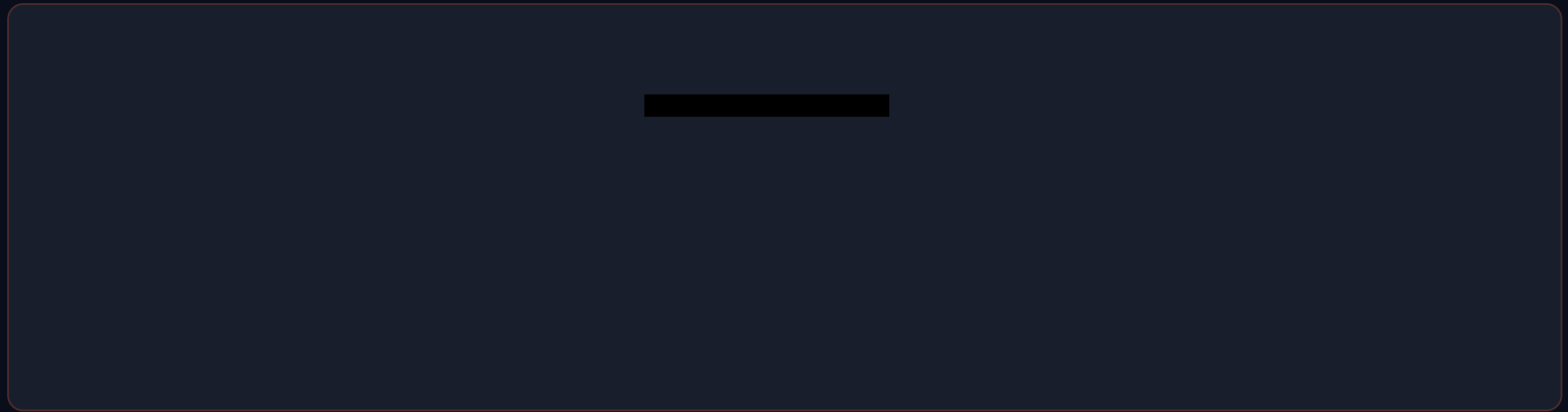
Goal: Learning agent, analytics dashboard, knowledge graph, and production optimisation.

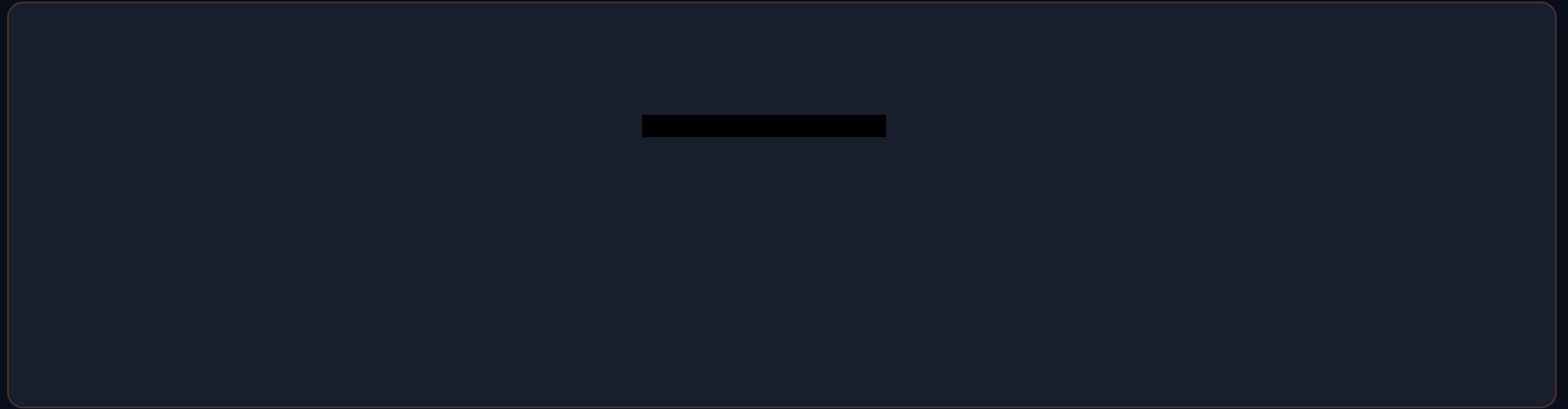
- Learning Agent with outcome-based strategy updates
- Reflection Agent (weekly meta-analysis)
- Analytics dashboard (Grafana + custom Next.js views)
- Knowledge Graph implementation (PostgreSQL graph queries)
- Memory Agent with five-tier hierarchy
- Error Recovery Agent with full circuit breaker suite
- Load testing and performance optimisation
- DR testing and runbook documentation
- Go-live and post-launch monitoring

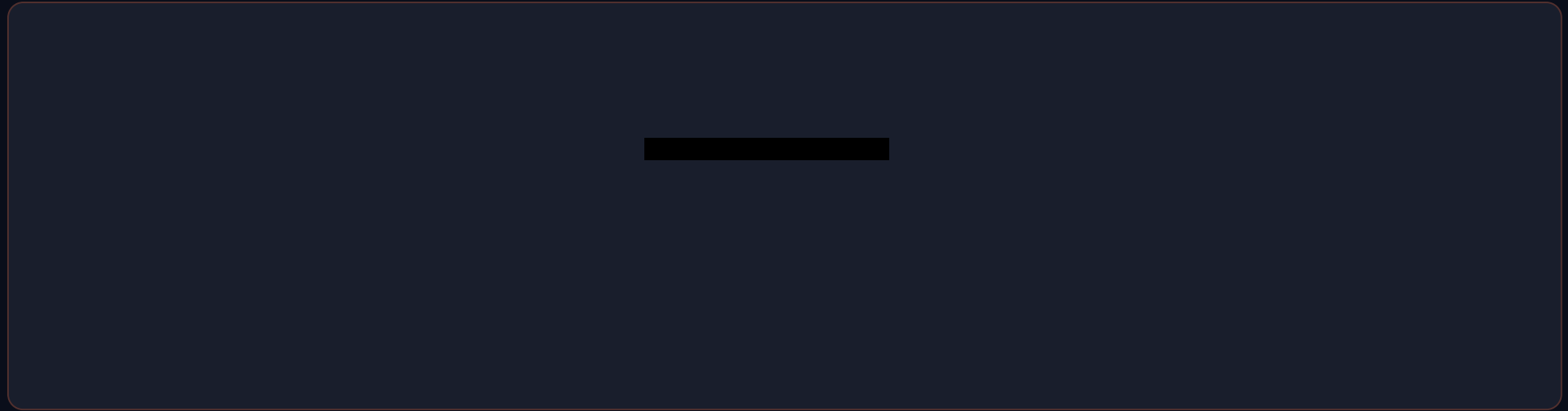
Gantt Chart

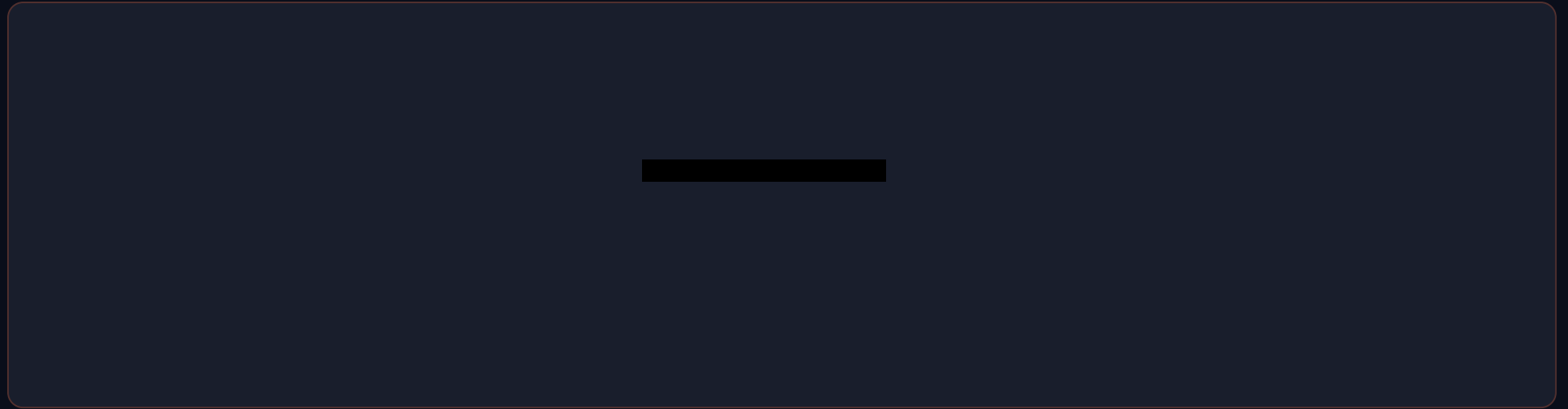






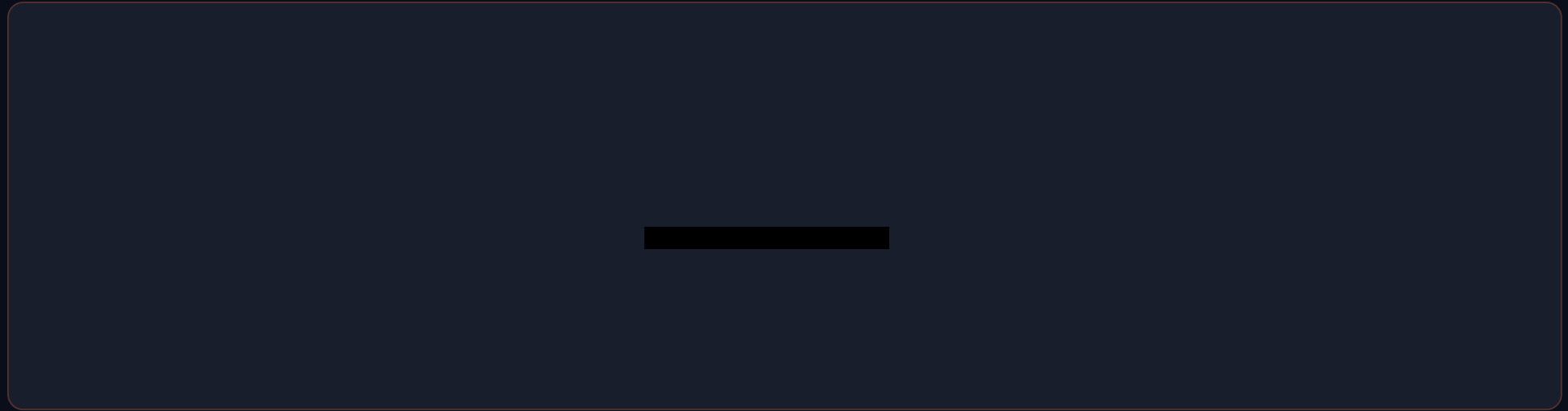


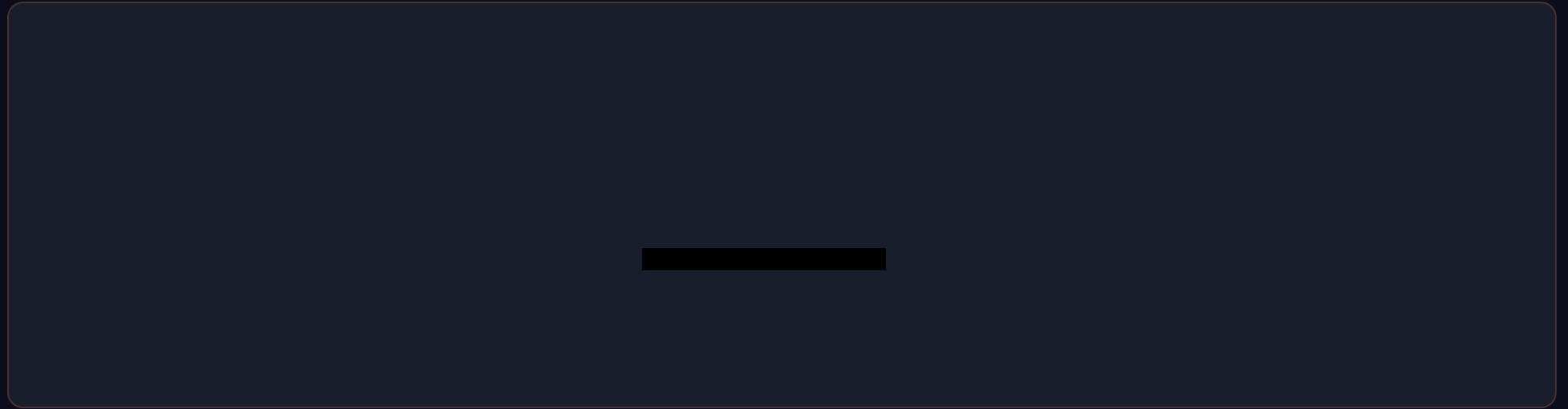


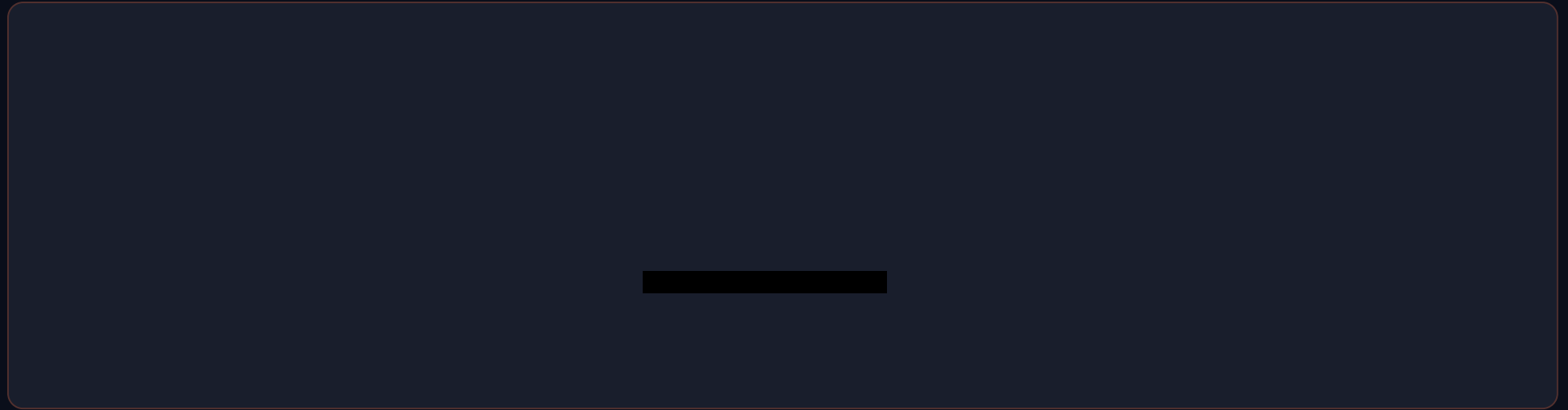


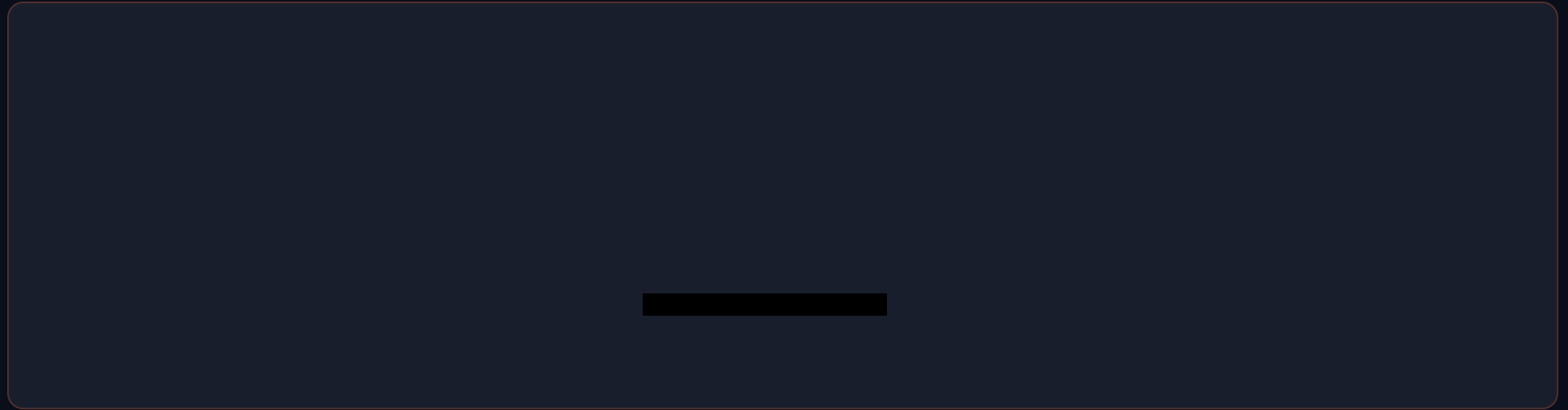


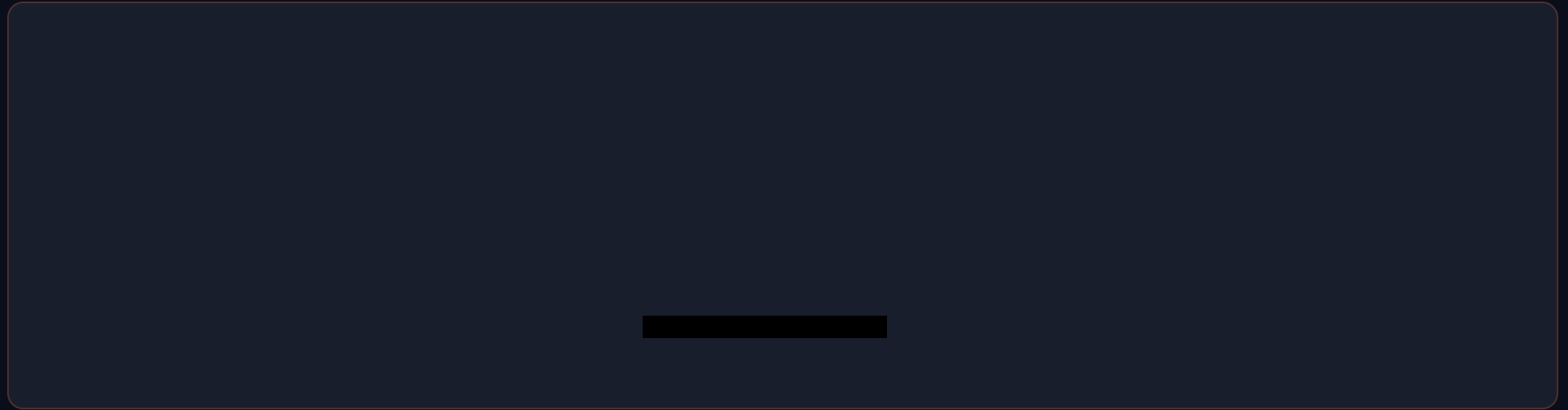














Risk Register

RISK	PROBABILITY	IMPACT	MITIGATION
ATS bot detection blocks Playwright automation	Medium	High	Playwright stealth mode, randomised delays, rate limiting, manual fallback UI
LLM hallucination reaches application output	Low	Critical	Verifier Agent + Compliance Agent dual-gate; evidence-only rewriting constraint
Resume formatting corruption on tailoring	Medium	High	Format preservation check Stage 8, automatic revert to base, never submit corrupt output
LinkedIn/SEEK rate limiting breaks discovery	High	Medium	Multi-source fallback (RSS, direct APIs, Google Jobs scraper), circuit breaker, caching

LLM provider outage (OpenAI)

Low

Medium

Multi-provider failover: GPT-4o → Claude 3.5 → Gemini Pro with identical prompt contracts

Decision Trees

Apply / Don't Apply Decision

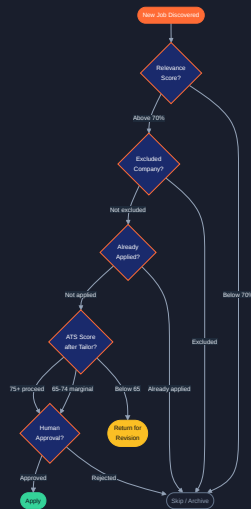


Figure 20 — Apply/Don't Apply Decision Tree.

Escalation Decision Tree

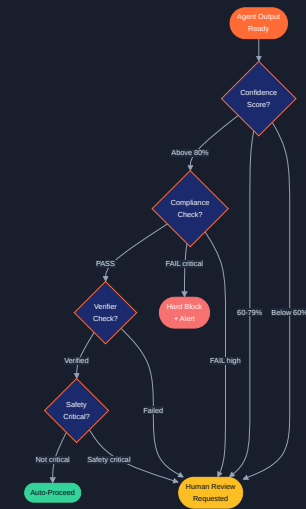


Figure 21 — Escalation Decision Tree: Confidence, Compliance, and Verification gates.

Resume Tailoring Depth Decision

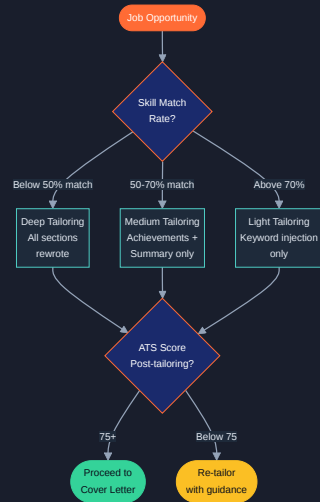


Figure 22 — Tailoring Depth Decision: matched to skill overlap rate.

State Machines

Application Lifecycle State Machine

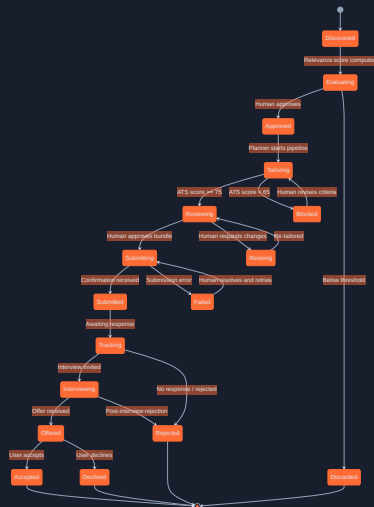


Figure 23 — Application Lifecycle: Full state machine from discovery through to resolution.

Agent Lifecycle State Machine

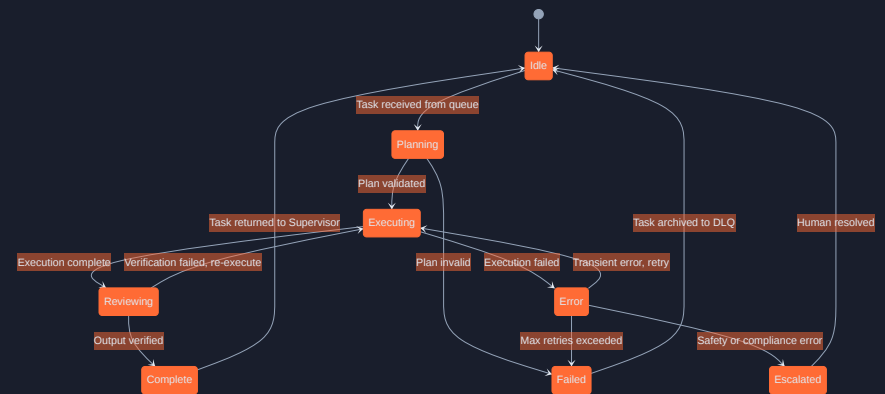


Figure 24 — Agent Lifecycle State Machine: Handles error recovery, escalation, and normal completion paths.

Appendix

Glossary of Terms

TERM	DEFINITION
ATS	Applicant Tracking System — software used by employers to manage job applications and screen candidates
C4 Model	Context, Container, Component, Code — a hierarchical software architecture documentation framework
Circuit Breaker	Design pattern that prevents repeated calls to a failing service by opening the circuit after a threshold of failures
DAG	Directed Acyclic Graph — a workflow graph with nodes (tasks) and directed edges (dependencies), with no cycles
DLQ	Dead Letter Queue — a queue holding messages that failed processing after maximum retry attempts
Embedding	A numerical vector representation of text, enabling semantic similarity comparison via cosine distance
Evidence Library	Structured store of verified professional evidence from portfolio, GitHub, and employment records

Human-in-the-Loop	System design pattern requiring human approval at critical decision points before autonomous action
JD	Job Description — the employer's documented requirements, responsibilities, and qualifications for a role
LangGraph	Python library for building stateful multi-agent workflows as directed graphs with LLM nodes
pgvector	PostgreSQL extension adding vector data types and similarity search operators (cosine, L2, inner product)
PII	Personally Identifiable Information — data that can identify an individual (name, phone, TFN, etc.)
PKCE	Proof Key for Code Exchange — OAuth2 extension preventing authorization code interception attacks
RAG	Retrieval-Augmented Generation — LLM pattern combining document retrieval with generation for grounded responses
RBAC	Role-Based Access Control — permission model assigning capabilities based on user roles
STAR Format	Situation, Task, Action, Result — structured method for framing professional achievements in interviews
TF-IDF	Term Frequency-Inverse Document Frequency — statistical measure of keyword importance in a document corpus

**Zero
Trust**

Security model requiring verification of every request regardless of network origin or user identity

Portfolio Evidence Catalog

The following verified evidence assets are available in Vikram Deshpande's portfolio and GitHub, forming the foundation of the evidence library that powers every application:

PROJECT	SOURCE	KEY EVIDENCE
JARVIS — Jira Analytics Dashboard	GitHub + Portfolio	Real-time LangChain pipeline; Python, TypeScript, Next.js, Supabase; live telemetry at 60 FPS
ATO Evidence Harness	Portfolio + LinkedIn	92% effort reduction (3hrs to 15min); COBOL/REXX automation; PCOMM, SDSF, PowerShell; ATO Payday Super program
Multi-Agent Orchestrator	GitHub	3-tier plan/execute/verify architecture; LangGraph; Python + React; demonstrates this document's design in miniature
Vedic Astronomy Platform	Portfolio + YouTube	AI/ML birth time rectifier; 3D celestial sphere simulations; GLSL shaders; WebSockets
AI Image Enhancer	GitHub	ML super-resolution; 2x scale detail restoration; Python; computer vision pipeline

LLM Evaluation Stack	Portfolio + Langfuse	38% error budget reduction; Langfuse + Phoenix; hallucination scoring; real production LLM pipeline
Cloud Modernisation (ANZ)	Resume / LinkedIn	30% delivery efficiency gain; 15% infrastructure cost reduction; Azure core banking; \$5M+ portfolio; 40+ resources
WebSocket Telemetry (ANZ)	Resume + Portfolio	10,000+ concurrent devices; P95 latency <200ms; real-time performance monitoring at scale

Document Revision History

VERSION	DATE	AUTHOR	CHANGES
v1.0	July 2026	Vikram Deshpande	Initial release — complete architecture package for review board

Document Authority: This architecture document is authored by Vikram Deshpande (Senior TPM / AI Solutions Architect) for internal review and planning purposes. All referenced portfolio projects, metrics, and achievements are verifiable through the sources cited in the Portfolio Evidence Catalog above. For verification, visit forgotten-mistory.web.app or GitHub @Victordtesla24.

Contact

CHANNEL	ADDRESS
Portfolio	forgotten-mistory.web.app
GitHub	@Victordtesla24
LinkedIn	vikramdeshpande.io
YouTube	@vicd0ct
Location	Melbourne, VIC, Australia